



IA PARA VALORACIÓN DE OPCIONES: MODELOS DE VOLATILIDAD EXPLICABLES

PEDRO GALLEGO LÓPEZ Y DANIEL RECUERO CORDOBÉS

Trabajo Final de Máster

Máster en Inteligencia Artificial y Computación Cuántica
Aplicada a los Mercados Financieros

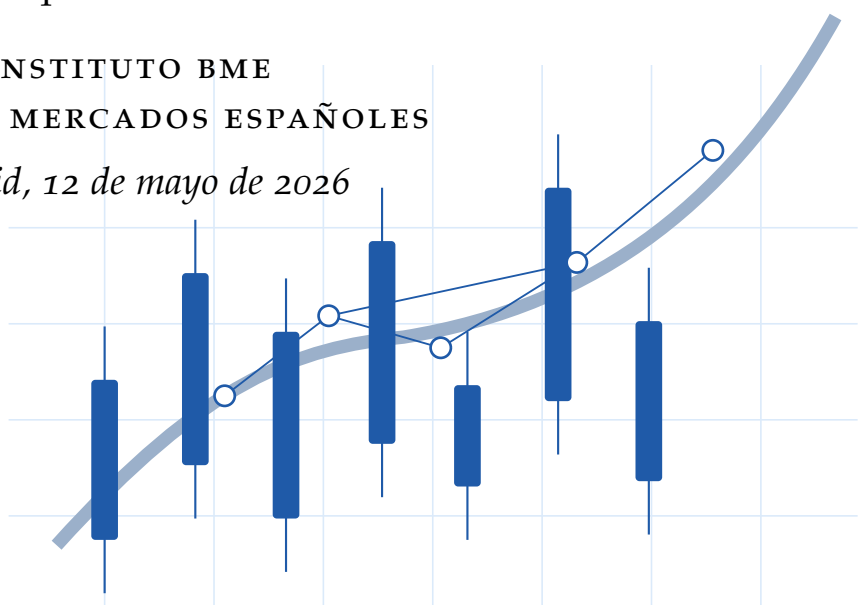
Dirección

Enrique Castellanos

INSTITUTO BME

BOLSAS Y MERCADOS ESPAÑOLES

Madrid, 12 de mayo de 2026



IA para valoración de opciones: modelos de volatilidad explicables

Pedro Gallego López
Daniel Recuero Cordobés

Palabras clave:

Volatilidad implícita, opciones IBEX, Black-76, aprendizaje automático, IA explicable, SHAP, dashboard, API, cloud

Resumen

Este Trabajo Final de Máster tiene como objetivo principal construir modelos de volatilidad implícita para opciones sobre el IBEX que sean tan fiables y potentes como los modelos modernos de inteligencia artificial, pero tan explicables como los modelos matemáticos tradicionales usados en finanzas cuantitativas. El trabajo parte de una tensión central: los modelos de IA pueden capturar relaciones no lineales complejas en la superficie de volatilidad, mientras que los modelos clásicos ofrecen una estructura interpretable y auditable. La propuesta busca unir ambas propiedades en una solución práctica.

A partir de operaciones reales, contratos, futuros subyacentes y series de tipos EONIA/€STR, se construye una base de volatilidad implícita mediante validaciones financieras e inversión numérica del modelo Black-76. Sobre esta base se entrenan varias familias de modelos de regresión: regresión lineal, Random Forest, XGBoost, redes neuronales y una arquitectura *quantum inspired*, inspirada en conceptos de computación cuántica. La evaluación no se limita al error predictivo, sino que incorpora estabilidad temporal, control de sobreajuste, coherencia financiera de la superficie y análisis regional por moneyness y vencimiento.

La contribución principal consiste en convertir modelos predictivos en sistemas explicables, auditables y consultables mediante un dashboard. La explicabilidad se aborda con SHAP global y local, superficies de volatilidad, curvas ICE y ALE, árboles surrogate, regresión simbólica, vecinos históricos y diagnósticos de residuales. Esta combinación permite justificar las predicciones con un lenguaje cercano al de los modelos tradicionales, sin renunciar a la capacidad de aproximación de la IA. El resultado es una arquitectura reproducible orientada a model validation, auditoría interna y uso operativo en entornos financieros.

Adicionalmente se ha habilitado una API para que cualquier persona pueda correr y usar estos modelos

AI for option pricing: explainable volatility models

Pedro Gallego López
Daniel Recuero Cordobés

Keywords:

Implied volatility, IBEX options, Black-76, machine learning, explainable AI, SHAP, dashboard, API, cloud

Abstract

The main objective of this Master's Thesis is to build implied-volatility models for IBEX options that are as reliable and powerful as modern artificial intelligence models, while remaining as explainable as traditional mathematical models used in quantitative finance. The work addresses a central tension: AI models can capture complex nonlinear relationships across the volatility surface, whereas classical models provide structure, interpretability and auditability. The proposed solution aims to combine both properties in a practical modelling framework.

Starting from real trades, contracts, underlying futures and EONIA/€STR rate series, the project builds an implied-volatility database through financial validations and numerical inversion of the Black-76 pricing model. Several regression families are then trained, including linear regression, Random Forest, XGBoost, neural networks and a quantum-inspired architecture, based on quantum computing concepts as its name suggests. The evaluation is not restricted to predictive error; it also considers temporal stability, overfitting control, financial coherence of the surface, and regional behaviour across moneyness and maturity.

The main contribution is to turn predictive models into explainable, auditable and queryable systems through a dashboard and a FastAPI service. Explainability is addressed through global and local SHAP, volatility surfaces, ICE and ALE curves, surrogate trees, symbolic regression, historical neighbours and residual diagnostics. This combination makes it possible to justify predictions in a language close to traditional financial models, without giving up the approximation power of AI. The resulting architecture is reproducible and oriented towards model validation, internal audit and operational use in financial environments.

Additionally, an API has been enabled so that anyone can run and use these models.

ÍNDICE GENERAL

1. INTRODUCCIÓN	6
1.1. Planteamiento del problema	6
1.2. Relevancia financiera y profesional	7
1.3. Antecedentes	8
1.4. Objetivos	9
1.5. Contribución del TFM	9
1.6. Estructura de la memoria	10
2. MARCO TEÓRICO	12
2.1. Opciones, futuros y volatilidad implícita	12
2.2. Smile, moneyness y estructura temporal	13
2.3. Aprendizaje automático para volatilidad	14
2.4. Explicabilidad en modelos financieros	15
2.5. Infraestructura cloud y reproducibilidad	16
2.6. Métricas de error, estabilidad y fidelidad	17
2.7. Gobierno de modelos y preguntas de validación	18
3. METODOLOGÍA	20
3.1. Diseño general del estudio	20
3.2. Datos y muestra analizada	21
3.3. Validaciones y tratamiento financiero	26
3.4. Variables y feature engineering	26
3.5. Familias de modelos	31
3.6. Splits temporales y selección	33
3.7. Reentrenamiento y progressive ATM	34
3.8. Construcción del dashboard	38
3.9. API, almacenamiento y cloud	38
3.10. Reproducibilidad y trazabilidad	46
3.11. Arquitectura del repositorio	49
3.12. Plan de validación funcional	51
3.13. Riesgos metodológicos y controles	52
4. RESULTADOS Y DISCUSIÓN	54
4.1. Resultados	54
4.1.1. Comparativa general de modelos	54
4.1.2. Entrenamiento estándar frente a progressive ATM	55
4.1.3. Selección del modelo final	56
4.1.4. Diagnóstico de rendimiento	57
4.1.5. Behaviour And Surface	57
4.1.6. Explicabilidad global	58

4.1.7.	Modelos equivalentes: árboles y regresión simbólica	59
4.1.8.	Explicabilidad local y soporte histórico	60
4.2.	Discusión	60
4.2.1.	Lectura financiera	60
4.2.2.	Lectura de IA y Data Science	61
4.2.3.	Lectura de ingeniería y cloud	62
4.2.4.	Limitaciones	62
5.	CONCLUSIONES	63
A.	FUNDAMENTOS DE SHAP	65
B.	CURVAS ICE	69
C.	CURVAS ALE	71
D.	ÁRBOLES SURROGATE Y REGRESIÓN SIMBÓLICA	74
E.	ENTRENAMIENTO PROGRESSIVE ATM	78

INTRODUCCIÓN

1.1 PLANTEAMIENTO DEL PROBLEMA

La volatilidad implícita es una magnitud central en mercados de derivados. En una opción negociada, el precio observable resume expectativas, oferta y demanda, liquidez, microestructura y percepción de riesgo. Sin embargo, para comparar contratos con strikes y vencimientos distintos, los profesionales no suelen razonar únicamente en precios: transforman esos precios en volatilidades implícitas y analizan smiles, skews y estructuras temporales [Hull, 2022, Derman and Miller, 2016].

El problema de este trabajo surge de una tensión habitual en finanzas cuantitativas. Por un lado, los modelos de aprendizaje automático pueden aproximar relaciones no lineales entre strike, subyacente, vencimiento, tipo de interés y volatilidad. Por otro lado, en un entorno financiero no basta con obtener una predicción numéricamente competitiva. El modelo debe poder auditarse: hay que saber dónde falla, qué variables explican sus predicciones, si respeta patrones razonables de la superficie y si una predicción concreta está respaldada por datos históricos similares.

En este contexto, el proyecto construye modelos de volatilidad explicable, consultable y útil vía API. La explicabilidad se entiende en dos niveles. A nivel de ingeniería, sirve para detectar sesgos, errores de datos, incoherencias, discontinuidades y comportamientos no deseados. A nivel financiero y profesional, permite justificar decisiones ante revisiones internas, auditorías, model validation o análisis de riesgo.

Operativamente, el problema se concreta en cuatro preguntas.

- Cómo transformar operaciones reales de mercado en una base de volatilidad implícita que sea consistente, trazable y reutilizable.
- Cómo aproximar la superficie $\sigma(K, T)$ con modelos suficientemente flexibles sin introducir sesgos temporales o dependencias espurias.
- Cómo exponer ese modelo de forma utilizable, de manera que no quede encerrado en notebooks o tablas estáticas.

- Cómo saber cuándo una predicción es defendible y cuándo conviene tratarla con cautela porque está en una región poco soportada o mal explicada.

La memoria se construye alrededor de esa cadena completa. No se limita a entrenar un predictor, sino que conecta datos de mercado, inversión de Black-76, selección de familias, explicabilidad global y local, diagnóstico regional y despliegue. Ese enfoque es coherente con el tipo de uso que tendría el sistema en un entorno profesional: consulta repetida, revisión crítica y necesidad de justificar el resultado más allá de una métrica agregada.

1.2 RELEVANCIA FINANCIERA Y PROFESIONAL

La volatilidad implícita condensa información de mercado y se utiliza en valoración, cobertura, gestión de riesgos y análisis de escenarios. En opciones sobre índices, la forma de la superficie de volatilidad puede reflejar demanda de protección, asimetría percibida, presión de liquidez o cambios de régimen. Un modelo que suaviza en exceso las alas, que comprime volatilidades extremas o que presenta saltos artificiales en vencimientos cortos puede producir señales engañosas aunque su RMSE agregado sea aceptable.

La literatura clásica de derivados establece que la volatilidad no debe interpretarse como un simple parámetro estadístico fijo, sino como una magnitud inferida del precio bajo un modelo de valoración [Black, 1976, Hull, 2022]. La literatura específica sobre smile muestra que los mercados organizan la volatilidad en torno a moneyness y vencimiento, no solo en torno a niveles absolutos de strike [Derman and Miller, 2016]. Por ello, el proyecto se diseña desde la geometría financiera de la superficie: el aprendizaje automático se usa para aproximar una función de volatilidad, pero la evaluación exige interpretar su forma.

La relevancia práctica aparece en varios frentes.

- **Valoración y marking:** una estimación coherente de volatilidad implícita permite comparar contratos y revisar precios fuera de rango.
- **Gestión de riesgos:** la sensibilidad del error por moneyness y vencimiento importa tanto como el error medio total.
- **Gobierno de modelos:** el área usuaria necesita entender si el modelo reacciona con lógica financiera y no solo si ajusta bien una muestra histórica.
- **Infraestructura analítica:** si el modelo no puede consultarse, versionarse y explicarse, su valor operativo cae de forma drástica.

Desde esa perspectiva, un buen resultado no es únicamente “predecir bien”. También implica poder distinguir entre una predicción bien soportada, una extrapolación razonable y una salida que requiere revisión adicional. Ese matiz es el que justifi-

ca que la memoria dedique espacio tanto a finanzas como a IA y a arquitectura de servicio.

1.3 ANTECEDENTES

La valoración de opciones sobre futuros se apoya tradicionalmente en Black-76 [Black, 1976]. En ese marco, el precio de una opción europea depende del futuro subyacente, el strike, el vencimiento, el tipo de interés y la volatilidad. Cuando el precio de mercado se observa, la volatilidad implícita se obtiene resolviendo numéricamente la ecuación inversa.

En paralelo, el uso de modelos de aprendizaje automático para superficies financieras responde a la necesidad de capturar no linealidades, interacciones y patrones empíricos que no quedan completamente descritos por modelos paramétricos simples. Sin embargo, la adopción profesional de modelos flexibles exige herramientas de interpretación. SHAP proporciona una semántica aditiva para atribuir predicciones a variables [Lundberg and Lee, 2017]; ICE y ALE ayudan a estudiar respuestas funcionales [Goldstein et al., 2015, Apley and Zhu, 2020]; los árboles surrogate y la regresión simbólica aproximan modelos complejos mediante objetos interpretables [Cranmer, 2023].

Los antecedentes relevantes convergen en tres líneas.

- **Línea financiera:** la inversión de modelos como Black-76 convierte precios observados en volatilidades comparables y lleva el problema al terreno de la superficie.
- **Línea empírica:** los smiles y las estructuras temporales muestran que la volatilidad de mercado no es constante y que la relación con strike y vencimiento es no lineal.
- **Línea metodológica:** los modelos de caja negra ganan flexibilidad, pero obligan a complementar la predicción con mecanismos de explicación y validación.

Este trabajo se sitúa precisamente en la intersección de esas tres líneas. Toma la formulación clásica de derivados como base para construir la variable objetivo, utiliza familias de aprendizaje automático sobre datos tabulares para aproximar la función de volatilidad y añade una capa de explicabilidad y servicio para hacer la solución defendible en un contexto profesional.

1.4 OBJETIVOS

El objetivo general es desarrollar una solución reproducible para estimar, explicar y consultar volatilidad implícita de opciones del IBEX. Este objetivo se concreta en los siguientes objetivos específicos:

1. Construir una base de volatilidad implícita a partir de operaciones, contratos, futuros subyacentes y curvas de tipos, con validaciones de calidad y trazabilidad.
2. Diseñar un conjunto de variables financieras coherentes con la geometría de la superficie de volatilidad.
3. Entrenar y comparar varias familias de modelos de regresión bajo una metodología temporal que reduzca lookahead bias y data snooping.
4. Generar componentes de explicabilidad global, local, funcional y de diagnóstico.
5. Implementar una API y un dashboard que permitan consultar predicciones, explicar muestras concretas y revisar el comportamiento del modelo.
6. Preparar infraestructura cloud reproducible para almacenar modelos, paquetes explicativos y metadatos de servicio.

Tomados en conjunto, estos objetivos cubren los tres planos que estructuran la memoria. El primero es **financiero**, porque la variable objetivo y las transformaciones deben respetar la lógica de opciones sobre futuros. El segundo es **de IA y Data Science**, porque la comparación entre familias exige un diseño temporal riguroso, métricas correctas y lectura crítica del error. El tercero es **de ingeniería**, porque el resultado final no es un gráfico aislado, sino un sistema consultable y versionable.

1.5 CONTRIBUCIÓN DEL TFM

La principal contribución del TFM es integrar en un único flujo operativo elementos que habitualmente se tratan por separado: construcción de datos de mercado, inversión de Black-76, entrenamiento de modelos, explicabilidad, diagnóstico financiero, API y despliegue cloud. La Figura 1 resume esta cadena con las mismas dependencias que la documentación técnica.

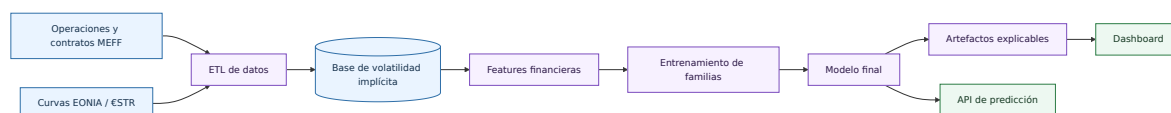


Figura 1: Vision general del proyecto.

La aportación académica se sitúa en la metodología de modelización y validación; la aportación profesional, en la capacidad de convertir un modelo de volatilidad en una herramienta inspeccionable y desplegable.

De forma más concreta, la contribución del trabajo puede resumirse así:

- **Contribución financiera:** construir una base de volatilidad implícita a partir de datos reales enlazando operaciones de opciones, futuros y curvas de tipos.
- **Contribución metodológica:** comparar familias heterogéneas con el mismo protocolo temporal, las mismas métricas y el mismo criterio de selección.
- **Contribución de explicabilidad:** combinar atribuciones SHAP, curvas ICE y ALE, modelos equivalentes, superficies y vecinos históricos en un único flujo de lectura.
- **Contribución de ingeniería:** dejar el modelo convertido en un paquete autocontenido consumible por dashboard y API, con almacenamiento reproducible y despliegue desacoplado.

Esa combinación es la que hace que el proyecto no sea sólo un ejercicio de entrenamiento de modelos. El valor añadido está en que cada capa técnica responde a una pregunta concreta y útil: cómo se construye la volatilidad, cómo se estima, cómo se valida, cómo se explica y cómo se sirve.

1.6 ESTRUCTURA DE LA MEMORIA

El [chapter 2](#) presenta los fundamentos financieros, de aprendizaje automático, explicabilidad e infraestructura. El [chapter 3](#) describe la construcción de datos, variables, modelos, validación, dashboard, API y cloud. El [chapter 4](#) analiza las métricas persistentes y la lectura que debe hacerse desde las vistas del dashboard. El [chapter 5](#) resume conclusiones y líneas futuras. Los anexos desarrollan SHAP, ICE, ALE, explicadores equivalentes y entrenamiento progressive ATM con mayor profundidad.

El orden de lectura responde al flujo real del proyecto.

1. En la **Introducción** se delimita el problema y se justifica por qué la explicabilidad es parte del resultado, no un añadido posterior.
2. En el **Marco Teórico** se fijan las piezas conceptuales necesarias para interpretar Black-76, smiles, familias de modelos, métricas y gobierno de modelos.
3. En la **Metodología** se documenta la implementación efectiva: ETL, features, validación temporal, reentrenamiento, generación del paquete de dashboard y despliegue.
4. En **Resultados y Discusión** se conectan métricas, superficies, explicaciones y limitaciones con una lectura financiera y técnica unificada.

5. En los **Anexos** se amplían los mecanismos de explicabilidad y entrenamiento para que la memoria principal conserve foco sin perder profundidad técnica.

MARCO TEÓRICO

2.1 OPCIONES, FUTUROS Y VOLATILIDAD IMPLÍCITA

Una opción financiera otorga a su comprador el derecho, pero no la obligación, de comprar o vender un activo subyacente a un precio de ejercicio K en una fecha futura. En el caso de opciones sobre futuros, el subyacente relevante para valoración es el precio del futuro F . La literatura de derivados enfatiza que el valor de una opción depende de la distribución esperada del subyacente y, de forma operativa, de la volatilidad usada en el modelo de valoración [Hull, 2022].

En el modelo Black-76, una call europea sobre un futuro se valora como:

$$C = e^{-rT} (FN(d_1) - KN(d_2)), \quad (1)$$

donde

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \quad (2)$$

Para una put:

$$P = e^{-rT} (KN(-d_2) - FN(-d_1)). \quad (3)$$

En estas expresiones, r es el tipo de interés, T el tiempo a vencimiento en años, σ la volatilidad y $N(\cdot)$ la función de distribución normal estándar. La volatilidad implícita σ_{imp} es el valor de σ que iguala el precio teórico al precio observado en mercado:

$$P_{mercado} = P_{Black76}(F, K, T, r, \sigma_{imp}). \quad (4)$$

La ecuación (4) no se resuelve cerradamente en general; se obtiene mediante métodos numéricos. En el proyecto se emplea bisección dentro de límites configurados, descartando filas no resolubles o no finitas.

Para el proyecto, esta formulación tiene varias implicaciones directas.

- La calidad de la volatilidad implícita depende de la calidad del precio de opción y del emparejamiento con el futuro subyacente.
- El tiempo a vencimiento no es un mero metadato: entra de forma no lineal a través de $\sqrt{T}$, por lo que errores en fechas o vencimientos contaminan la variable objetivo.
- El tipo de interés afecta tanto al descuento como a la construcción de variables derivadas basadas en el forward ajustado.
- Calls y puts comparten el mismo concepto de volatilidad implícita, pero pueden vivir en regiones distintas de la superficie y, por tanto, inducir respuestas diferentes en el modelo.

Por eso la memoria no trata la volatilidad implícita como una etiqueta estadística cualquiera. Antes de entrenar ningún modelo, la variable objetivo debe quedar bien definida desde el punto de vista financiero y bajo un procedimiento numérico reproducible.

2.2 SMILE, MONEYNES Y ESTRUCTURA TEMPORAL

Si la volatilidad fuese constante, opciones con distinto strike y vencimiento compartirían una misma σ . En mercados reales aparece una superficie de volatilidad:

$$\sigma_{imp} = \sigma(K, T), \quad (5)$$

o, de forma más comparable entre niveles de mercado:

$$\sigma_{imp} = \sigma(m, T), \quad m = \frac{F}{K}. \quad (6)$$

La moneyness m indica la posición relativa del subyacente frente al strike. Su transformación logarítmica,

$$\ell = \log(F/K), \quad (7)$$

es especialmente útil porque centra la región ATM en $\ell = 0$ y trata de forma más simétrica regiones a ambos lados del strike. La literatura sobre smile subraya que la curvatura y pendiente de la volatilidad implícita contienen información sobre

expectativas de mercado, asimetría y demanda de cobertura [Derman and Miller, 2016].

En la práctica, esta superficie se lee a través de varios patrones.

- **Smile o skew:** describe cómo cambia la volatilidad implícita al alejarse de ATM.
- **Estructura temporal:** muestra cómo cambia la volatilidad con el vencimiento para una región dada de moneyness.
- **Interacción entre ambas:** la curvatura del smile no tiene por qué ser la misma en corto plazo que en vencimientos largos.

Esta lectura geométrica justifica el diseño posterior de variables y visualizaciones. Si el fenómeno financiero se organiza en torno a moneyness y vencimiento, es razonable que el feature engineering, la validación y los gráficos de comportamiento del modelo se construyan también alrededor de esas magnitudes.

2.3 APRENDIZAJE AUTOMÁTICO PARA VOLATILIDAD

El problema de modelización se formula como una regresión supervisada:

$$\hat{\sigma} = f(X), \quad (8)$$

donde X contiene variables financieras observables y transformadas. El proyecto compara familias con diferentes niveles de flexibilidad:

- Regresión lineal, como baseline interpretable y control de complejidad.
- Random Forest, basado en agregación de árboles para capturar interacciones no lineales [Breiman, 2001].
- XGBoost, como boosting regularizado con early stopping y buen rendimiento en datos tabulares [Chen and Guestrin, 2016].
- Redes neuronales densas, como aproximadores flexibles de funciones no lineales [Hornik et al., 1989].
- Una red neuronal clásica *quantum inspired*, que introduce una estructura tensorial inspirada en ideas de representación compacta. No se presenta como computación cuántica real, sino como una variante arquitectónica implementada en código.

La comparación no puede basarse solo en error medio. En datos financieros temporales, una evaluación mal planteada puede introducir fuga de información. Por ello se emplean splits temporales, control de contratos compartidos y k-folds temporales, tal como se desarrolla en el [section 3.6](#).

La comparación entre familias se apoya en cuatro criterios.

- **Capacidad de ajuste:** qué nivel de no linealidad e interacción puede capturar cada familia.
- **Estabilidad temporal:** cómo se comporta el modelo al cambiar de fold o al pasar de validation a test.
- **Coste operativo:** si necesita escalado, early stopping, mayor presupuesto de entrenamiento o persistencia adicional.
- **Interpretación posterior:** hasta qué punto requiere apoyarse en SHAP, surrogates, superficies o vecinos para ser defendible.

Esa es la razón por la que el proyecto no enfrenta “modelos buenos” frente a “modelos malos”, sino familias con perfiles distintos. La regresión lineal aporta una referencia interpretable, los árboles capturan relaciones tabulares robustas, el boosting ofrece flexibilidad adicional y las redes permiten explorar arquitecturas más expresivas.

2.4 EXPLICABILIDAD EN MODELOS FINANCIEROS

La explicabilidad cumple una función técnica y una función de gobierno de modelos. Técnicamente, permite detectar variables dominantes, discontinuidades, extrapolaciones y errores locales. Profesionalmente, facilita responder por qué una predicción se considera razonable y en qué condiciones no debería confiarse en ella.

SHAP descompone la predicción de un modelo en un valor base y contribuciones por variable:

$$\hat{\sigma}(x) = \phi_0 + \sum_{j=1}^p \phi_j(x), \quad (9)$$

donde ϕ_j mide la contribución atribuida a la característica j [Lundberg and Lee, 2017]. En este proyecto se complementa con ICE, ALE, árboles surrogate, regresión simbólica y vecinos históricos. La razón es que ninguna herramienta por sí sola responde todas las preguntas relevantes: SHAP explica contribuciones, ICE muestra trayectorias individuales, ALE estima efectos acumulados locales, los surrogates dan reglas aproximadas y los vecinos informan sobre soporte local.

La lógica de uso de cada técnica es la siguiente:

- **SHAP:** qué variables empujan una predicción y qué drivers dominan globalmente el modelo.
- **ICE:** cómo cambia la salida del modelo cuando se mueve una variable para una observación concreta.
- **ALE:** qué efecto medio local aparece en regiones observadas, reduciendo el problema de contrafactuales poco plausibles.

- **Árboles surrogate y regresión simbólica:** si el comportamiento global del predictor puede resumirse mediante reglas o fórmulas más legibles.
- **Vecinos históricos:** si la observación explicada está respaldada por casos parecidos en la muestra de entrenamiento.

Esta combinación encaja especialmente bien en volatilidad implícita, porque el usuario no sólo pregunta “qué predice el modelo”, sino también “por qué”, “con qué soporte”, “con qué suavidad” y “en qué zona falla”.

2.5 INFRAESTRUCTURA CLOUD Y REPRODUCIBILIDAD

Un modelo útil en un contexto profesional necesita más que un notebook. Debe poder cargarse de forma reproducible, exponer predicciones, separar salidas de entrenamiento y servir explicaciones de manera consistente. Por ello el proyecto separa entrenamiento, construcción de paquetes explicativos, dashboard, API y almacenamiento de modelos. La infraestructura cloud no se incorpora como elemento decorativo, sino para resolver problemas operativos: portabilidad de servicios, centralización de modelos, cache local, despliegue independiente de API y dashboard y trazabilidad de versiones.

En términos operativos, eso implica al menos cinco requisitos.

- Persistir el modelo final junto con sus metadatos y, cuando aplica, su scaler.
- Generar un paquete de dashboard que pueda cargarse sin recalcular explicaciones costosas en cada arranque.
- Mantener desacoplados dashboard y API para poder desplegarlos y versionarlos por separado.
- Permitir backend local y backend GCP sin cambiar el contrato funcional de las aplicaciones.
- Conservar trazabilidad de configuración, rutas, versiones y recursos de infraestructura.

La reproducibilidad, por tanto, no se limita a “poder volver a entrenar”. También exige poder reconstruir qué versión del modelo respondió una consulta, qué componentes explicativos estaban asociados a esa versión y desde qué almacenamiento se cargaron.

En el contexto del TFM, esa reproducibilidad se apoya en varias capas coordinadas.

- **Capa de entrenamiento:** fija datos, parámetros, semillas y métricas para que el proceso pueda repetirse y compararse.
- **Capa de persistencia:** conserva modelo, scaler, metadatos y paquete explicativo como una unidad coherente.

- **Capa de servicio:** hace que API y dashboard lean exactamente la misma versión del modelo, sin reconstrucciones ad hoc.
- **Capa de infraestructura:** permite mover el sistema entre backend local y GCP manteniendo el mismo contrato funcional.

Sin esa separación, el riesgo no sería sólo operativo. También aparecería un problema metodológico: cualquier conclusión obtenida en la memoria podría dejar de ser reproducible en cuanto la versión desplegada divergiera de la versión analizada. Por eso la infraestructura forma parte del diseño científico del trabajo y no sólo de su empaquetado técnico.

2.6 MÉTRICAS DE ERROR, ESTABILIDAD Y FIDELIDAD

La evaluación de un modelo de volatilidad requiere métricas complementarias. El error absoluto medio,

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (10)$$

ofrece una lectura directa en unidades de volatilidad. Es robusto y fácil de comunicar, pero no penaliza de forma diferencial los errores grandes. Por ese motivo se utiliza también RMSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (11)$$

El RMSE es especialmente relevante en model validation porque amplifica errores extremos. En una superficie de volatilidad, esos errores pueden aparecer en vencimientos cortos, alas o regiones con menor liquidez. El coeficiente de determinación,

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (12)$$

resume proporción de variabilidad explicada, aunque debe interpretarse con prudencia en datos financieros heterogéneos.

Junto a estas métricas frente a mercado, el dashboard usa métricas de fidelidad para modelos explicables equivalentes. La fidelidad compara el surrogate con el modelo principal, no con la volatilidad real. Esta distinción es importante: un árbol surrogate puede ser muy fiel a un modelo sesgado, y un modelo principal puede tener buen RMSE pero ser difícil de resumir con reglas simples. Por tanto, la evaluación final

debe combinar tres planos: precisión frente a mercado, estabilidad del proceso de selección y fidelidad de las explicaciones.

Cada métrica responde a una pregunta distinta.

- **MAE:** cuánto se equivoca el modelo en promedio en unidades de volatilidad y con una lectura sencilla.
- **RMSE:** cuánto pesan los errores extremos, especialmente relevantes en alas o vencimientos cortos.
- R^2 : qué fracción de variabilidad consigue explicar el predictor sobre la muestra evaluada.
- **Métricas de fidelidad:** qué parte del comportamiento del modelo principal pueden reproducir los modelos equivalentes.

La memoria insiste en esta separación porque evita un error conceptual habitual: confundir una buena explicación del modelo con una buena explicación del mercado. Un surrogate puede ser útil para entender el predictor y, al mismo tiempo, ser insuficiente para justificar por sí solo la calidad financiera del resultado.

2.7 GOBIERNO DE MODELOS Y PREGUNTAS DE VALIDACIÓN

En un entorno financiero, un modelo de volatilidad debería poder responder a preguntas de model validation. Algunas son cuantitativas: cuál es el error fuera de muestra, si existe gap entre entrenamiento y validación, si el error cambia por vencimiento o moneyness, y si el resultado es estable entre folds. Otras son cualitativas pero igualmente relevantes: si el modelo reacciona de forma razonable ante cambios de strike, si produce superficies suaves donde el mercado debería ser continuo, si una predicción manual se encuentra dentro de una región observada o si el predictor depende excesivamente de una variable que no debería dominar.

El diseño del proyecto convierte esas preguntas en vistas concretas del dashboard. La pestaña de diagnóstico responde dónde falla. La pestaña de superficie responde cómo se comporta como función financiera. La explicabilidad global responde qué variables organizan las predicciones. La explicabilidad local responde por qué una muestra concreta obtiene un valor determinado y si existen observaciones históricas cercanas. Esta estructura facilita la defensa ante un tribunal porque alinea cada componente técnico con una pregunta profesional.

En consecuencia, el gobierno del modelo se apoya en una lectura guiada.

- Primero se revisa el **rendimiento agregado**: métricas, dispersión y comportamiento train/validation/test.
- Después se analiza el **rendimiento regional**: error por moneyness, error por maturity y mapas de residuales.

- A continuación se inspecciona el **comportamiento funcional**: superficies, smiles, term structures, ICE y ALE.
- Finalmente se revisa la **explicación causal aparente del modelo**: SHAP global, explicabilidad local, vecinos y modelos equivalentes.

Esta secuencia evita que una única visualización monopolice la interpretación. En un entorno de derivados, la confianza en una predicción surge de la coherencia entre varias vistas, no de una sola gráfica llamativa o de una única métrica sintética.

METODOLOGÍA

3.1 DISEÑO GENERAL DEL ESTUDIO

El estudio tiene carácter empírico y aplicado. Parte de datos de mercado, construye una variable objetivo financiera mediante inversión de Black-76, entrena modelos de regresión y evalúa tanto rendimiento predictivo como coherencia explicativa. La Figura 2 resume el flujo metodológico.

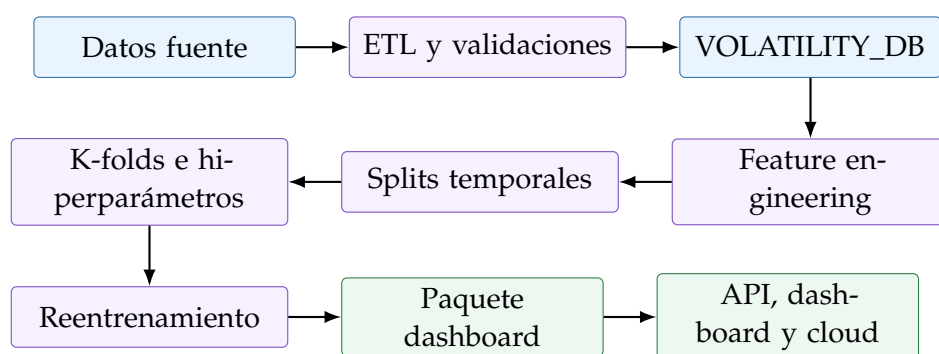


Figura 2: Flujo metodológico del TFM.

El flujo no es lineal sólo en apariencia. Cada bloque deja salidas persistidas y comprobables que sirven de entrada al siguiente, de modo que la metodología puede auditarse por etapas. Primero se consolida la base financiera; después se fija un espacio de variables coherente con la superficie; a continuación se comparan familias bajo un protocolo temporal común; y, por último, se transforma el modelo seleccionado en un sistema consultable.

Este diseño responde a cuatro principios metodológicos.

- **Separación de responsabilidades:** ETL, modelización, explicabilidad y servicio no se mezclan en un único bloque opaco.
- **Trazabilidad de extremo a extremo:** cada predicción puede rastrearse hasta datos, configuración y versión de modelo.

- **Comparabilidad entre familias:** todas las alternativas compiten bajo las mismas reglas de split, métricas y reentrenamiento.
- **Lectura profesional del resultado:** el producto final no es sólo una tabla de métricas, sino un modelo interpretable y servible.

Esta estructura es importante porque condiciona la credibilidad de los resultados. Si el entrenamiento fuese correcto pero las explicaciones o el servicio no reutilizasen exactamente la misma lógica de features, la calidad del sistema quedaría comprometida aunque las métricas numéricas fueran buenas.

3.2 DATOS Y MUESTRA ANALIZADA

El pipeline de datos transforma ficheros de contratos, operaciones y tipos en una base final de volatilidad implícita. La documentación MkDocs identifica tres fuentes principales: contratos, operaciones y series EONIA/€STR. Los contratos aportan información de código, strike y vencimiento; las operaciones aportan precio, cantidad, hora y contrato; los tipos permiten calcular el tipo compuesto aplicable hasta vencimiento.

La cadena ETL se materializa por pasos, como muestran las Figuras 3 y 4. Esta organización permite cachear salidas intermedias, validar contratos de datos y aislar responsabilidades: los *StepLoaders* orquestan lectura, cache y validación, mientras que los builders construyen la salida material de cada paso.

Los datos se filtran al dominio de opciones IBEX mensuales y futuros mensuales asociados. Las opciones semanales y contratos fuera de dominio se excluyen para mantener coherencia contractual. La salida final contiene fecha y hora de ejecución, código de opción, tipo call/put, precio negociado, strike, futuro subyacente, precio del subyacente, lag temporal, tiempo a vencimiento, tipo compuesto y volatilidad implícita.

El primer paso, *Read Raw*, normaliza ficheros de mercado con o sin cabecera, elimina columnas auxiliares no pertenecientes al esquema final, convierte fechas, horas y numéricos, filtra prefijos IBEX y construye una serie homogénea de tipos. La base de tipos combina EONIA ajustado por el spread configurado antes de la fecha de corte y €STR desde esa fecha. El segundo paso, *Merge Raw*, une operaciones y contratos por *ContractCode* y año de sesión, imputa vencimientos con la regla del tercer viernes e imputa strikes desde el tramo configurado del código cuando la fuente no lo trae completo.

El tercer paso, *Product Split*, divide la base unificada en opciones, futuros y relación opción-futuro por vencimiento. El cuarto, *Underlying*, resuelve el precio de futuro subyacente mediante una unión temporal *as-of*: para cada opción selecciona el último trade del futuro con hora menor o igual a la operación de la opción. Esta deci-

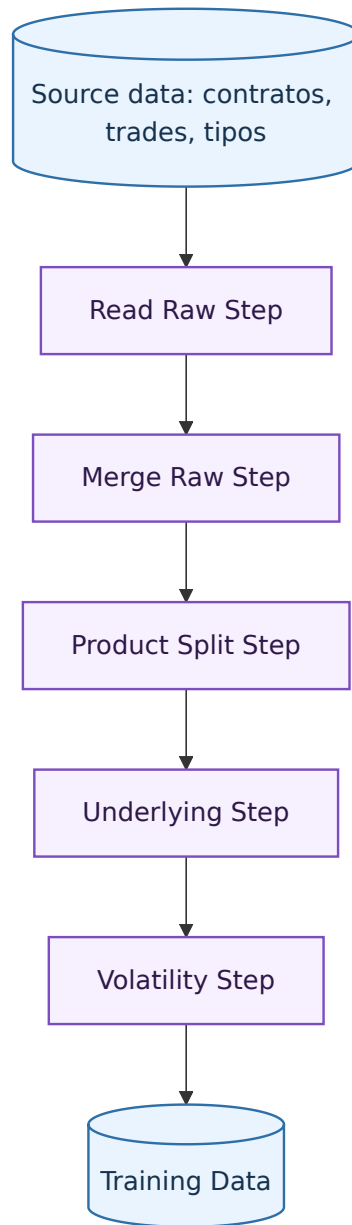


Figura 3: Secuencia principal de procesamiento de datos.

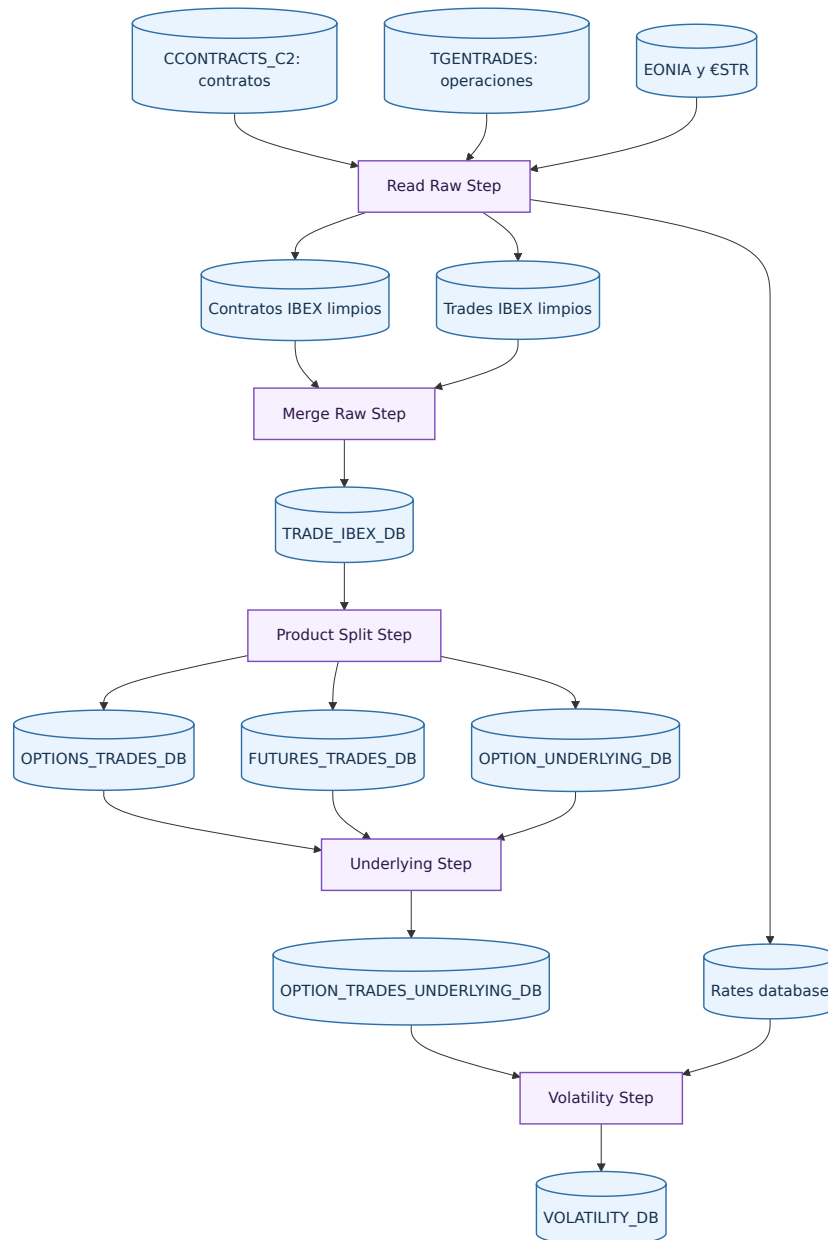


Figura 4: Pipeline completo de datos hasta VOLATILITY_DB.

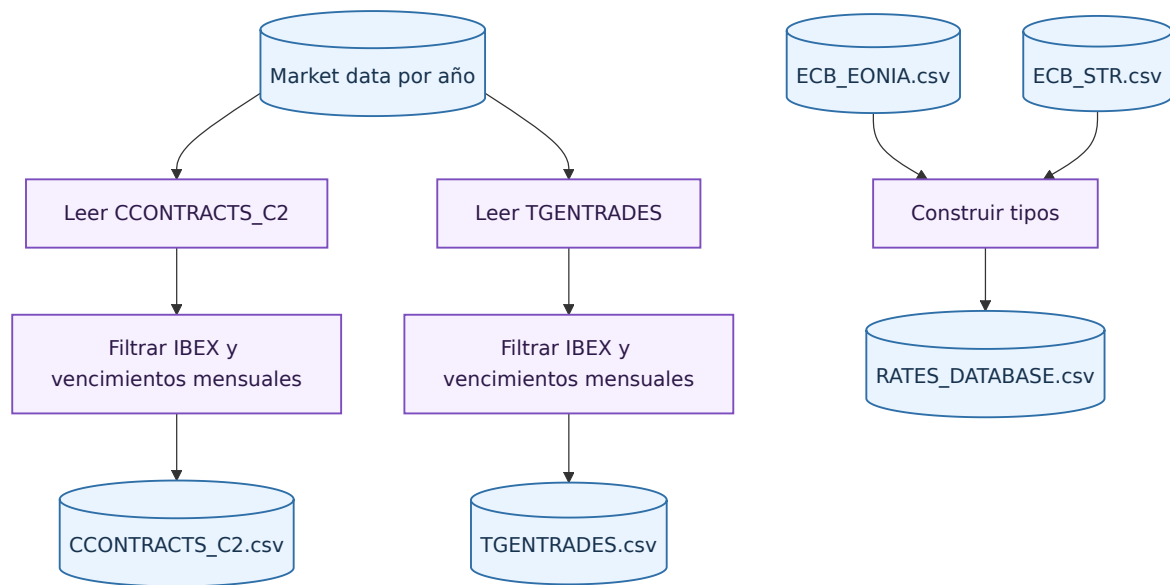


Figura 5: Lectura y normalización de datos fuente.

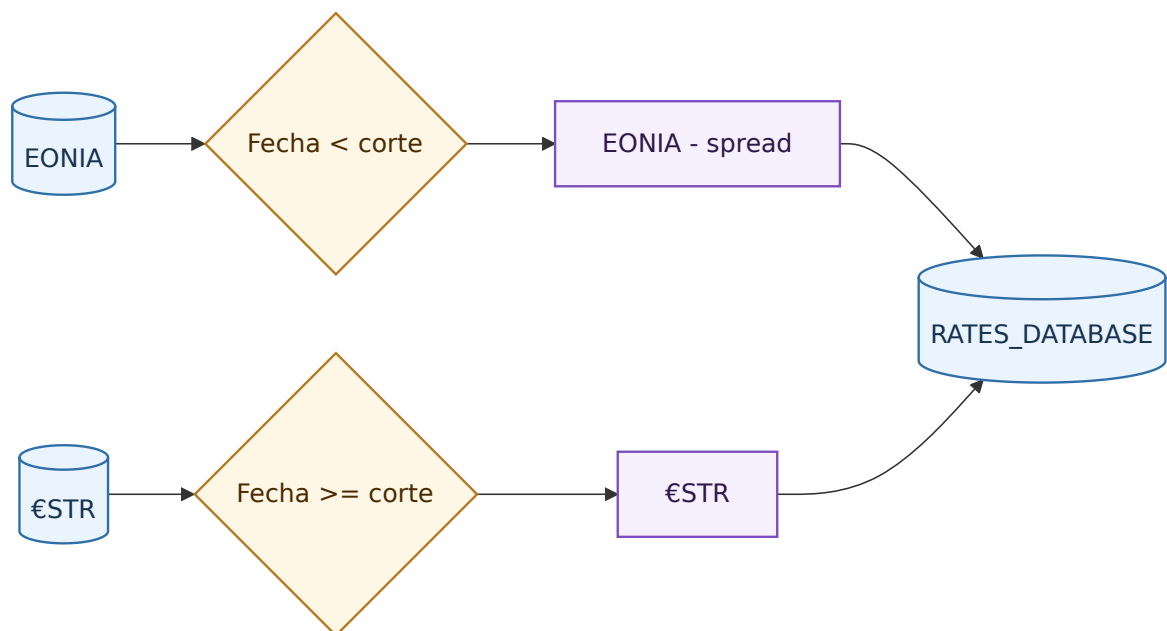


Figura 6: Construcción de la base homogénea de tipos.

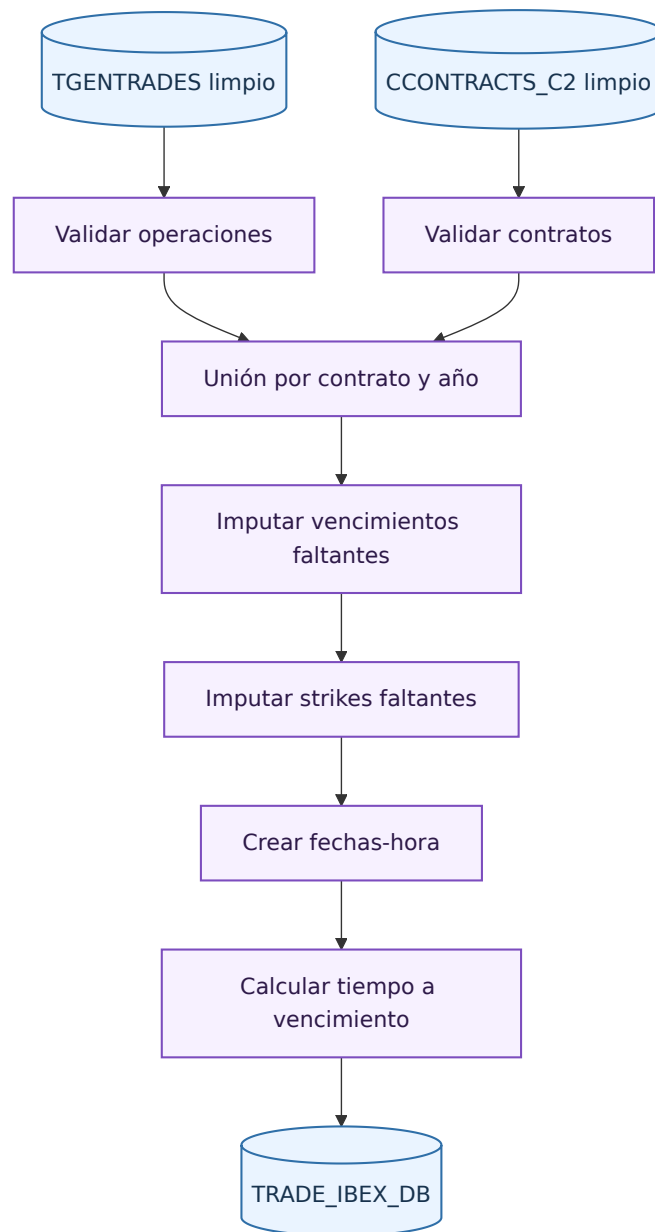


Figura 7: Union de operaciones con informacion contractual.

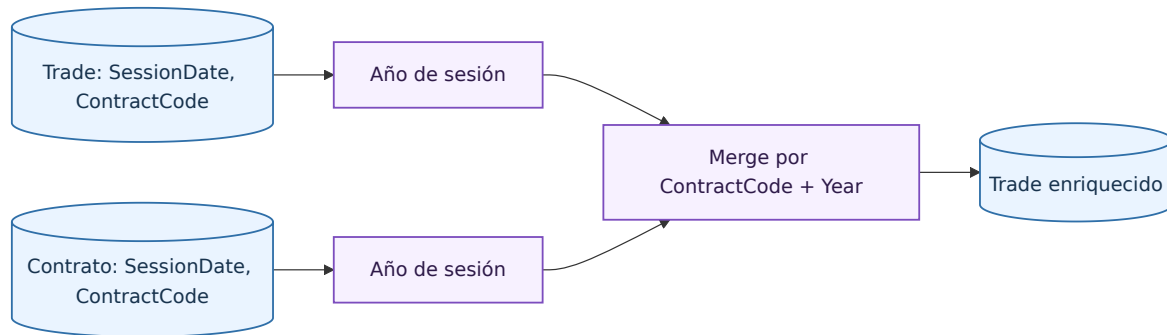


Figura 8: Union por codigo de contrato y ano de sesion.

sión evita lookahead dentro del propio ETL. El quinto, *Volatility*, filtra operaciones sin subyacente fiable o con lag excesivo, calcula el tipo compuesto hasta vencimiento y resuelve la volatilidad implícita por Black-76 inverso.

3.3 VALIDACIONES Y TRATAMIENTO FINANCIERO

El ETL aplica validaciones transversales: precios positivos, cantidades positivas, ausencia de claves duplicadas, control de missing values, vencimientos coherentes, strikes compatibles con el código de contrato y subyacente temporalmente anterior a la operación de la opción. Esta última condición es crítica para evitar utilizar información futura.

El cálculo de volatilidad implícita se realiza en el último paso. Se eliminan operaciones sin subyacente fiable, se calcula el tipo compuesto hasta vencimiento y se resuelve la ecuación (4). Si no existe cambio de signo en el intervalo de búsqueda o la solución es no finita, la observación se marca como no resoluble y no alimenta el entrenamiento.

3.4 VARIABLES Y FEATURE ENGINEERING

Las entradas raw visibles son *OptionType*, *StrikePrice*, *UnderlyingPrice*, *TimeToExpiration* y *Rate*. Estas variables son suficientes para reconstruir el vector de entrenamiento y también son comprensibles para un usuario financiero que consulte la API.

El vector transformado se diseña para representar la geometría de la superficie. La Tabla 1 resume las variables derivadas.

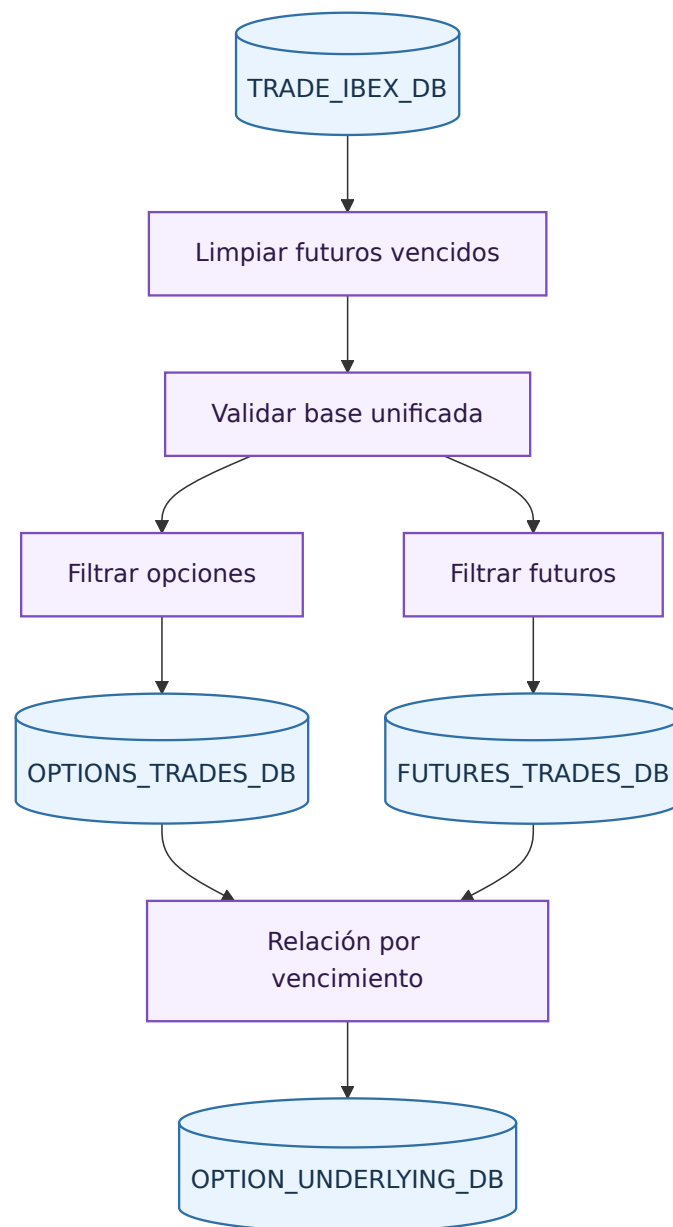


Figura 9: Separacion de operaciones por producto.

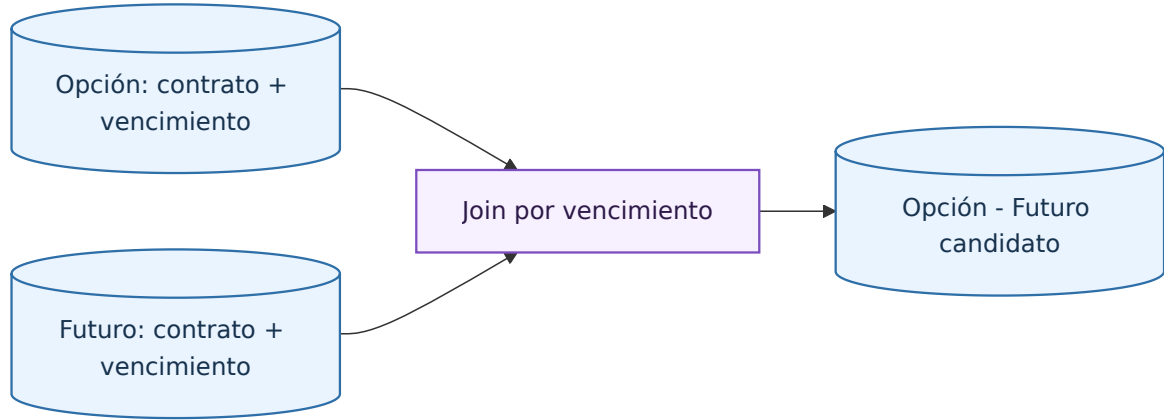


Figura 10: Relacion opcion-futuro por vencimiento.

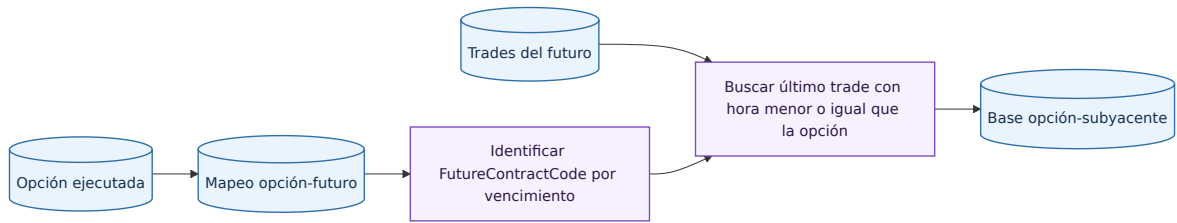


Figura 11: Asignacion de futuro subyacente a cada opcion.

Feature	Fórmula	Motivo
TTEYears	$T = T_{\text{días}}/365$	Vencimiento en años
sqrtTTEYears	$\sqrt{T}$	Escala temporal de Black-76
logMoneyness	$\log(F/K)$	Posición relativa frente al strike
logMoneynessSq	$\log(F/K)^2$	Curvatura del smile
logMoneynessXSqrtTTE	$\log(F/K)\sqrt{T}$	Interacción smile-vencimiento
logForwardMoneyness	$\log(Fe^{rT}/K)$	Ajuste por tipo y vencimiento
rate	r	Sensibilidad directa a tipos
isCall, isPut	Indicadores	Tipo de opción

Cuadro 1: Features financieras utilizadas por los modelos.

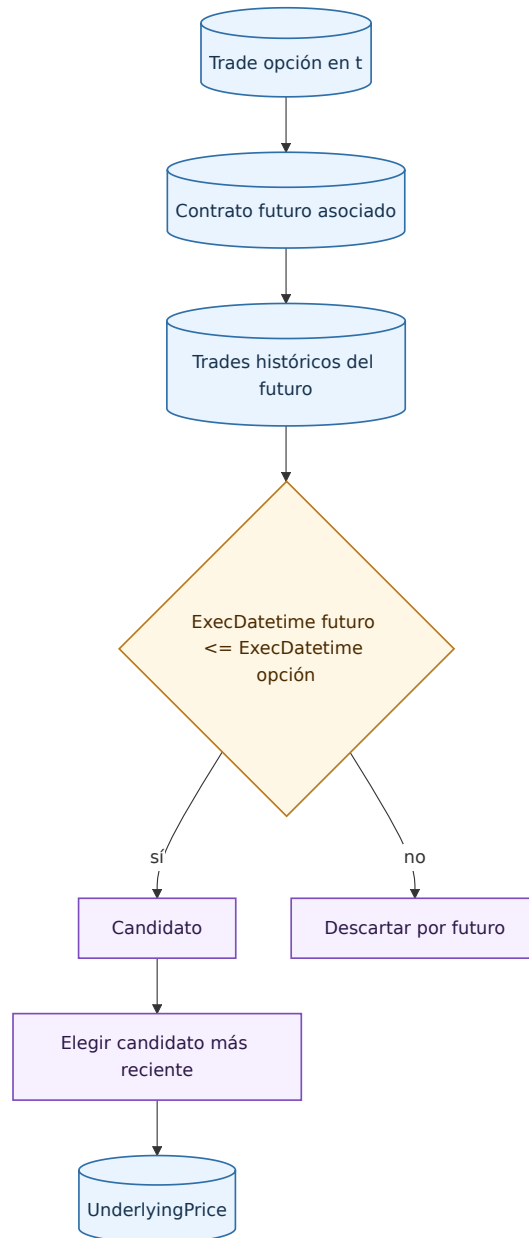


Figura 12: Selección temporal as-of del subyacente.

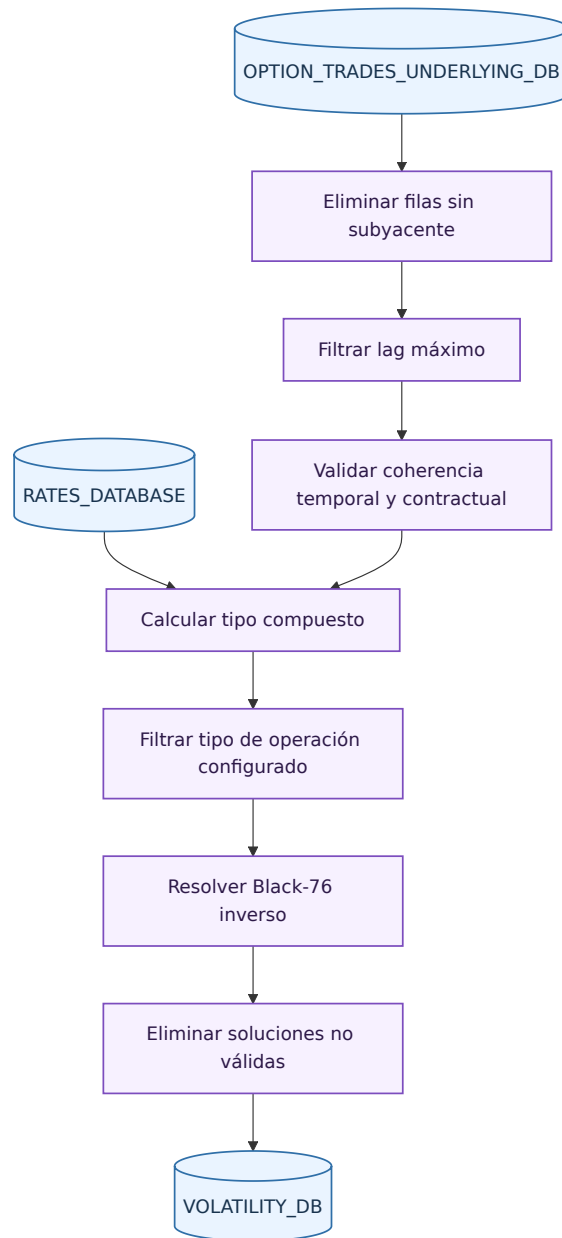


Figura 13: Calculo de volatilidad implicita.

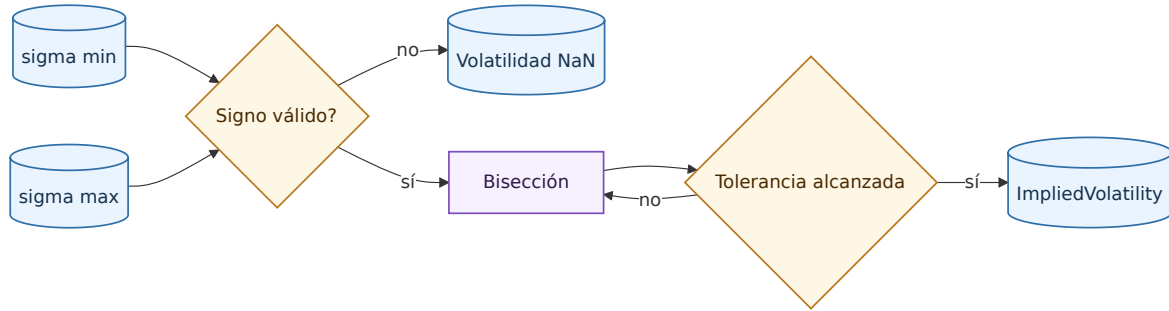


Figura 14: Solver por biseccion de la volatilidad implicita.

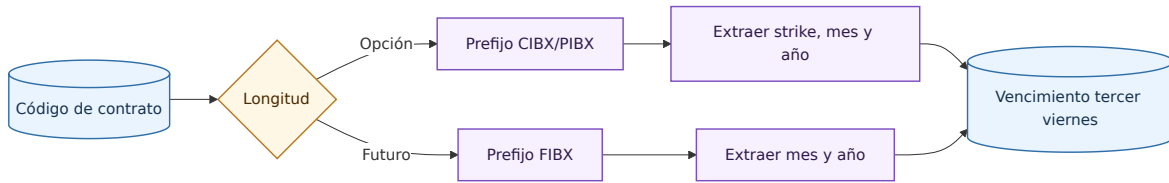


Figura 15: Validacion e interpretacion del codigo de contrato.

Esta elección evita que el modelo dependa de niveles absolutos de strike que pueden cambiar con el régimen de mercado. La moneyness permite comparar contratos negociados en momentos distintos del IBEX, mientras que el vencimiento y sus transformaciones capturan estructura temporal.

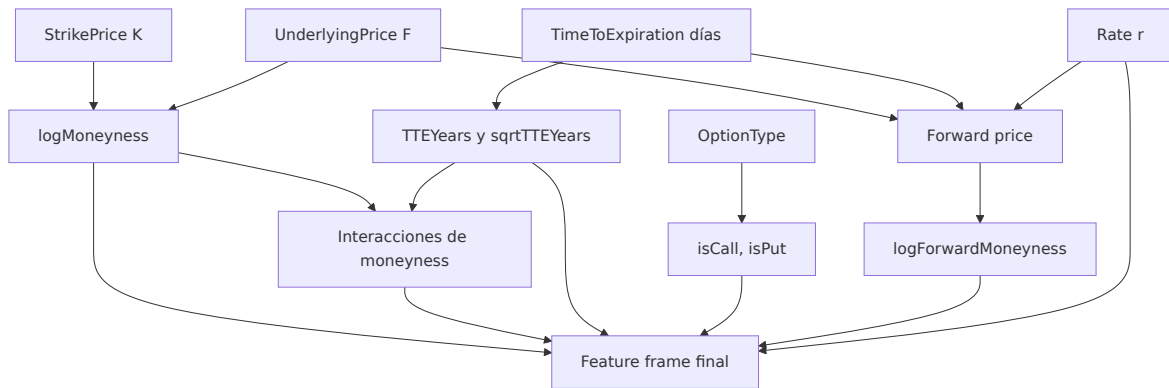


Figura 16: Construcción de features financieras.

3.5 FAMILIAS DE MODELOS

El repositorio implementa una abstracción común para familias de modelos. Cada familia declara nombre, parámetros fijos, espacio de búsqueda, instanciación, entrenamiento y persistencia. La Tabla 2 resume su papel en el estudio.

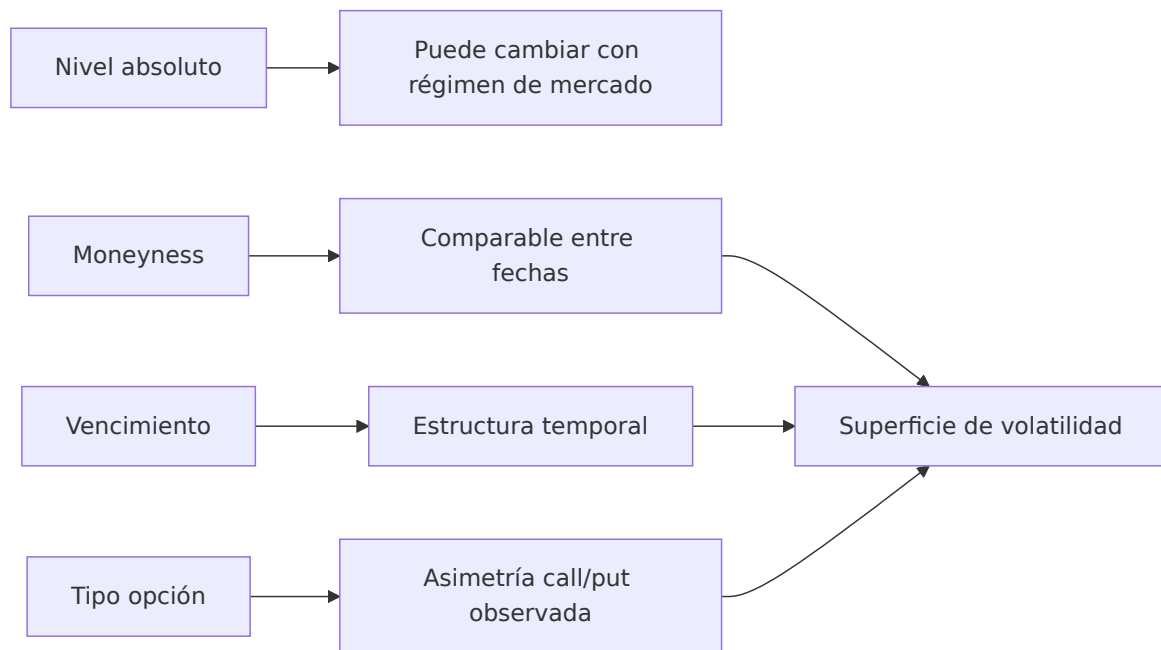


Figura 17: Motivacion financiera de las features.

Familia	Papel metodológico
Regresión lineal	Baseline interpretable para evaluar cuánto aporta la no linealidad.
Random Forest	Modelo tabular robusto ante interacciones y no linealidades sin escalado.
XGBoost	Boosting regularizado con early stopping y alta capacidad predictiva.
Red neuronal secuencial	Aproximador flexible con escalado de variables numéricas.
Quantum Inspired NN	Variante clásica de red neuronal con estructura tensorial inspirada en ideas cuánticas; se evalúa como arquitectura implementada, no como computación cuántica física.

Cuadro 2: Familias de modelos entrenadas en el proyecto.

Todas las familias heredan una interfaz común: declaran parámetros fijos, espacio de búsqueda, instanciación, entrenamiento, persistencia y número de configuraciones exploradas. Esta homogeneidad permite comparar un baseline lineal, modelos de árboles, boosting y arquitecturas neuronales con el mismo protocolo de selección. En la familia Quantum Inspired, el cálculo sigue siendo clásico: las features se proyectan a un embedding, se reordenan como tensor y se contraen mediante una capa tensor-train antes de una capa densa final.

La elección de familias responde a una lógica comparativa y no decorativa.

- La **regresión lineal** fija una referencia interpretable y permite medir cuánto valor añade la no linealidad una vez creadas las features correctas.
- **Random Forest** y **XGBoost** representan dos formas distintas de explotar interacciones tabulares complejas sin exigir el mismo tipo de preprocesado que las redes.
- Las **redes neuronales** sirven para comprobar si una arquitectura más flexible mejora realmente el problema o sólo incrementa el coste y la complejidad de explicación.
- La variante **Quantum Inspired** introduce una hipótesis arquitectónica diferenciada manteniendo computación clásica, lo que permite valorar su aportación real frente a alternativas más estándar.

La interfaz común del repositorio hace además que la comparación sea metodológicamente limpia. Todas las familias pasan por los mismos splits temporales, las mismas métricas y el mismo proceso de reentrenamiento y conversión a paquete de dashboard. Eso evita que una familia “gane” por haber sido evaluada con un protocolo distinto del resto.

3.6 SPLITS TEMPORALES Y SELECCIÓN

La validación sigue tres principios. Primero, el orden temporal se respeta: train precede a validation y test. Segundo, puede existir un lag de fechas entre bloques para reducir contaminación temporal. Tercero, se controla la exclusividad de contratos: si el mismo contrato aparece en más de un split, se conserva en un único bloque según volumetría y prioridad.

Esta regla es relevante porque una opción puede negociarse durante varios días. Si un contrato aparece en entrenamiento y validación, el modelo podría aprender rasgos idiosincráticos del contrato y ofrecer una estimación optimista. Las Figuras 22, 23, 24 y 25 resumen la lógica temporal, la separación con lag y el control de contratos compartidos.

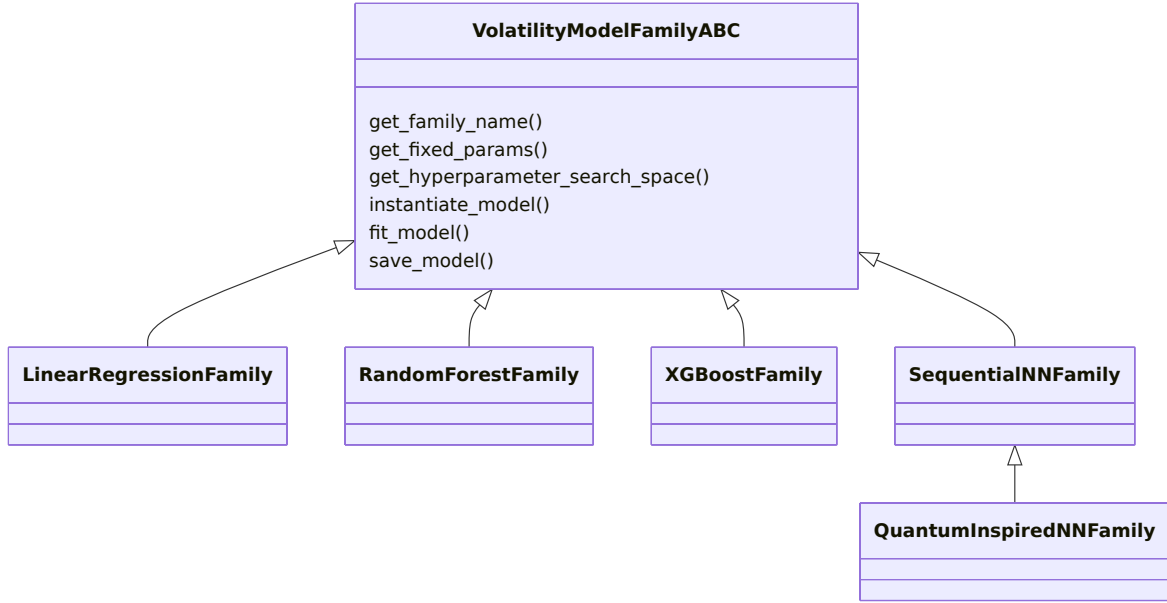


Figura 18: Abstraccion comun de familias de modelos.

La búsqueda de hiperparámetros se realiza con k-folds temporales. Para cada candidato se calculan MAE, RMSE y R^2 en train y validation. La selección usa una métrica compuesta:

$$CE_1 = RMSE_{val} + \alpha \cdot std(RMSE_{val}) + \beta \cdot \max(0, RMSE_{val} - RMSE_{train}). \quad (13)$$

Esta métrica penaliza error de validación, inestabilidad entre folds y gap de sobreajuste. Según la configuración del repositorio, $\alpha = 0,25$ y $\beta = 0,75$.

3.7 REENTRENAMIENTO Y PROGRESSIVE ATM

Después de seleccionar hiperparámetros, el modelo se reentrena en dos fases: train/validation y final test. En la fase final se ajusta con trainval y se evalúa contra test. La métrica de reentrenamiento incorpora el rendimiento observado en selección:

$$CE_2 = RMSE_{val} + \gamma \cdot CE_1 + \beta \cdot \max(0, RMSE_{val} - RMSE_{train}), \quad (14)$$

con $\gamma = 0,25$ y $\beta = 0,75$ en la configuración actual.

El proyecto incluye una variante progressive ATM. Ordena observaciones por $|\log(F/K)|$, de más ATM a alas, y divide el entrenamiento en segmentos. La idea

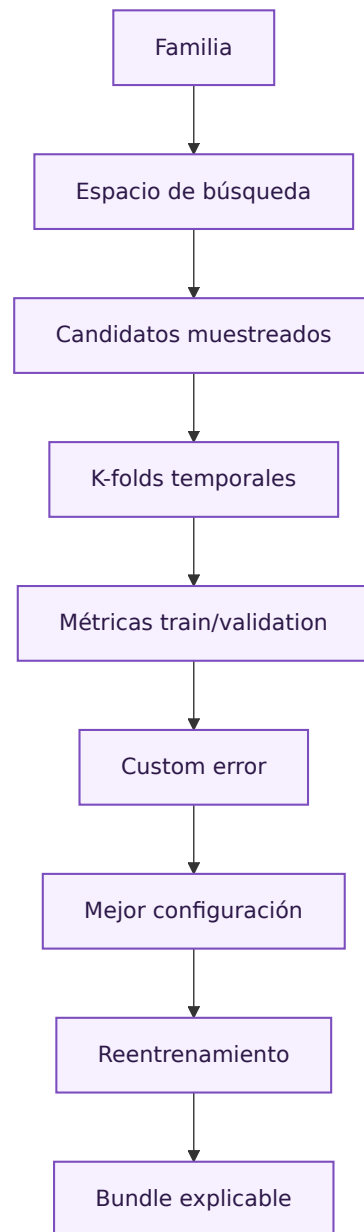


Figura 19: Selección de modelos mediante k-folds temporales.

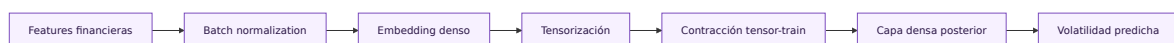


Figura 20: Arquitectura Quantum Inspired clásica basada en tensor-train.

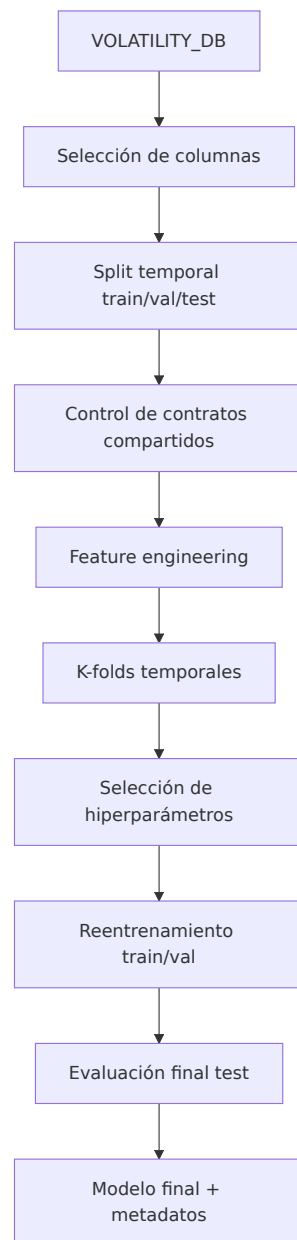


Figura 21: Flujo de modelización de volatilidad.

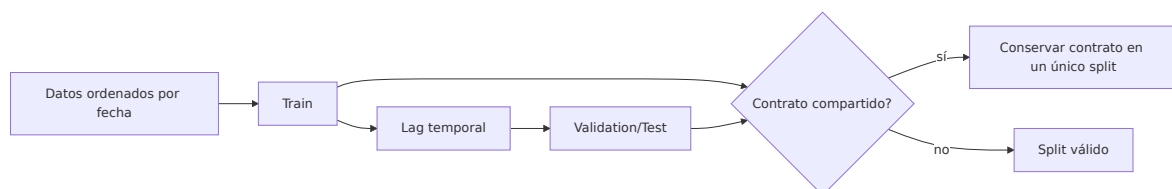


Figura 22: Control temporal y exclusividad de contratos.

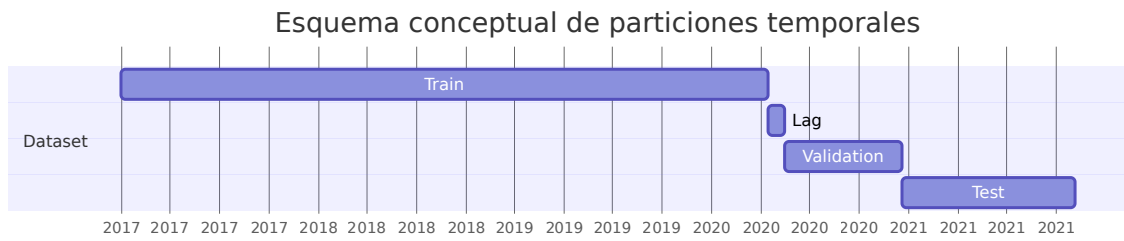


Figura 23: Esquema conceptual de particiones temporales.

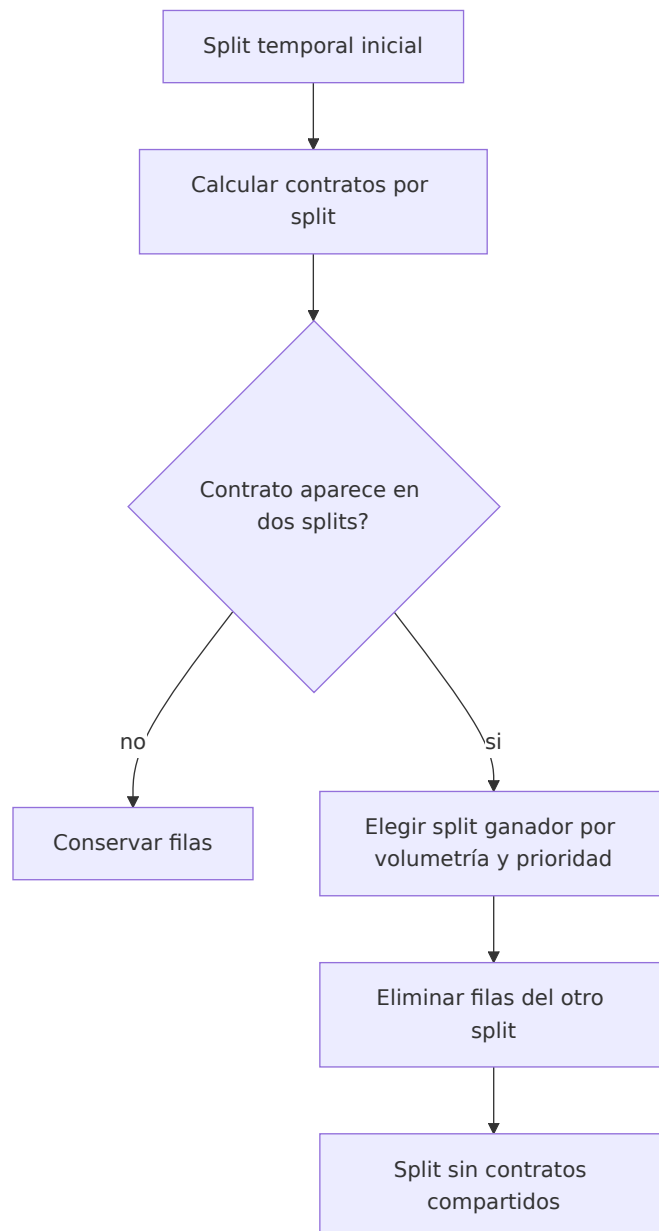


Figura 24: Control de contratos compartidos entre splits.

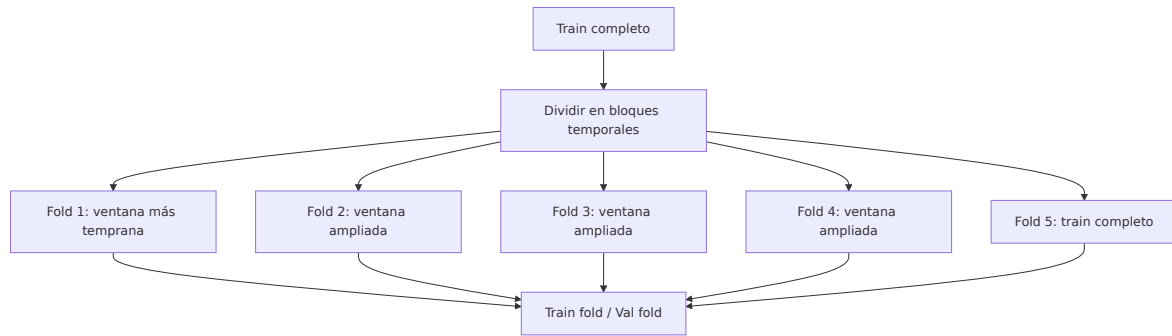


Figura 25: Construcción de k-folds temporales.

financiera es aprender primero la zona central, habitualmente más líquida y estable, y después incorporar regiones más alejadas. En modelos no iterativos se implementa mediante pesos; en XGBoost y redes se implementa mediante fases acumulativas. Por su carácter experimental, esta variante se menciona aquí y se desarrolla en el [chapter E](#).

3.8 CONSTRUCCIÓN DEL DASHBOARD

El dashboard no entrena modelos. Consume paquetes autocontenidos generados desde modelos finales. Cada paquete contiene dataset con predicciones, residuales y error absoluto, muestras para explicabilidad local, anclas de superficies, SHAP global y local, árboles surrogate, regresor simbólico, vecinos, superficies, ICE, ALE y diagnóstico agregado. La conversión desde modelo final a ese paquete no modifica el predictor: carga el runtime, aplica el mismo feature engineering sobre train y test, calcula explicaciones y guarda una carpeta autocontenida en `src/dashboard/saved_models/`.

La aplicación se estructura en cuatro pestañas, resumidas en la Tabla 3. Esta organización responde a preguntas distintas y evita confundir error predictivo con explicación.

3.9 API, ALMACENAMIENTO Y CLOUD

La API se implementa con FastAPI. El código expone endpoints de salud, predicción y explicabilidad local:

- `/health`
- `/run_model/predict/`
- `/run_model/sample_explainability/`

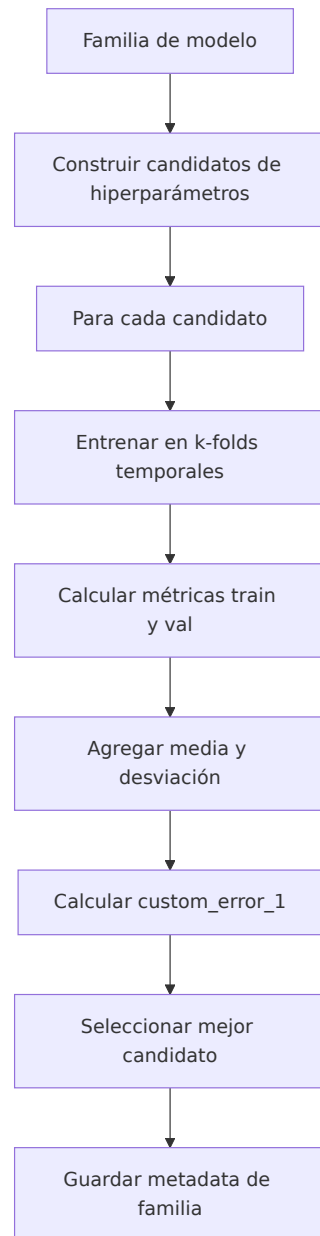


Figura 26: Búsqueda de hiperparametros por familia.

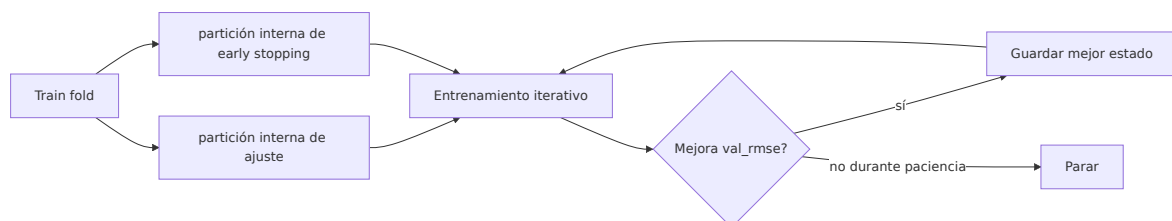


Figura 27: Early stopping con validacion interna.

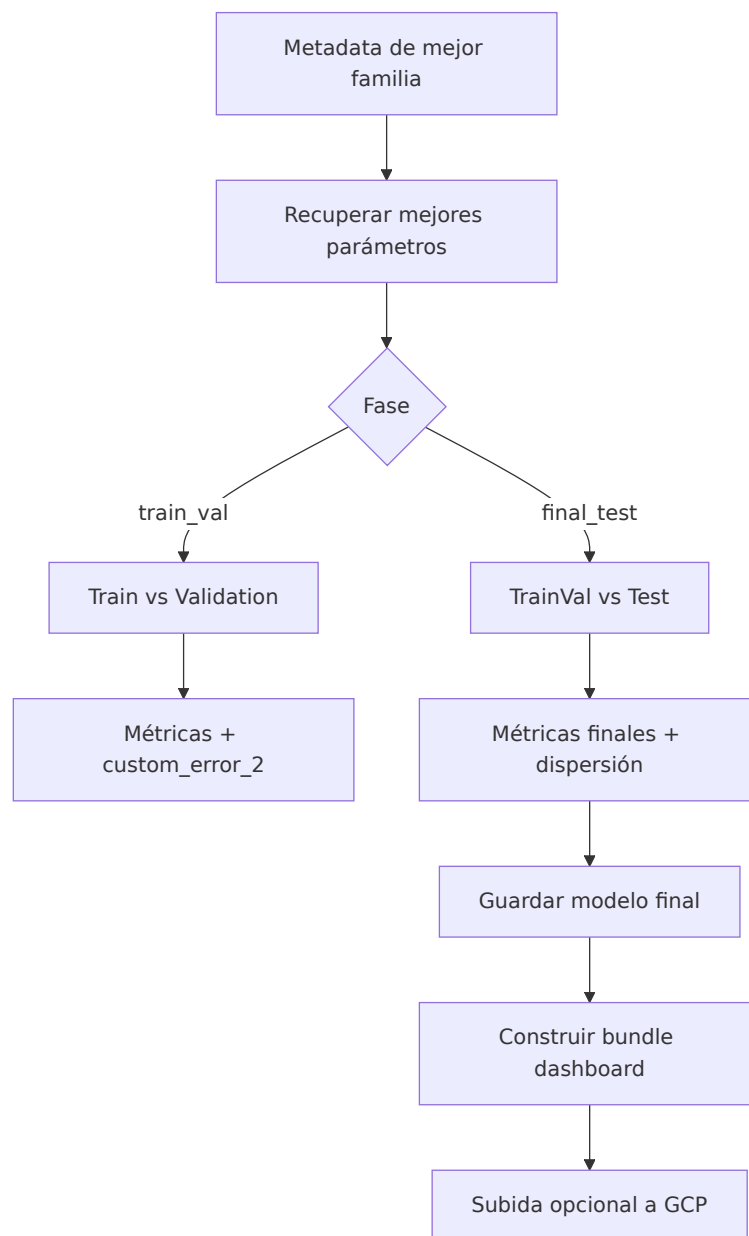


Figura 28: Reentrenamiento del mejor candidato.

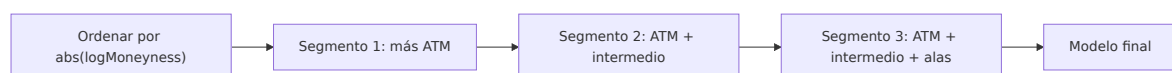


Figura 29: Entrenamiento progressive ATM por segmentos.

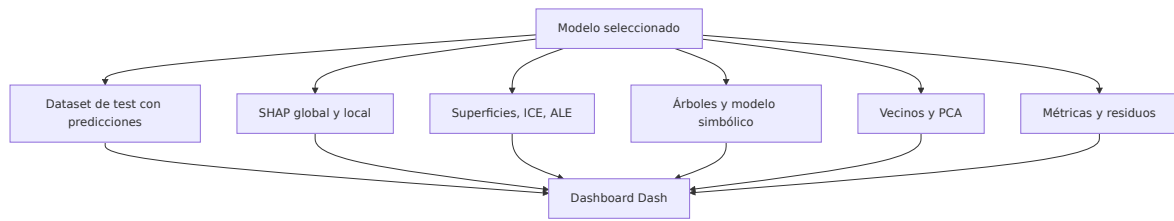


Figura 30: Entradas precomputadas consumidas por el dashboard.

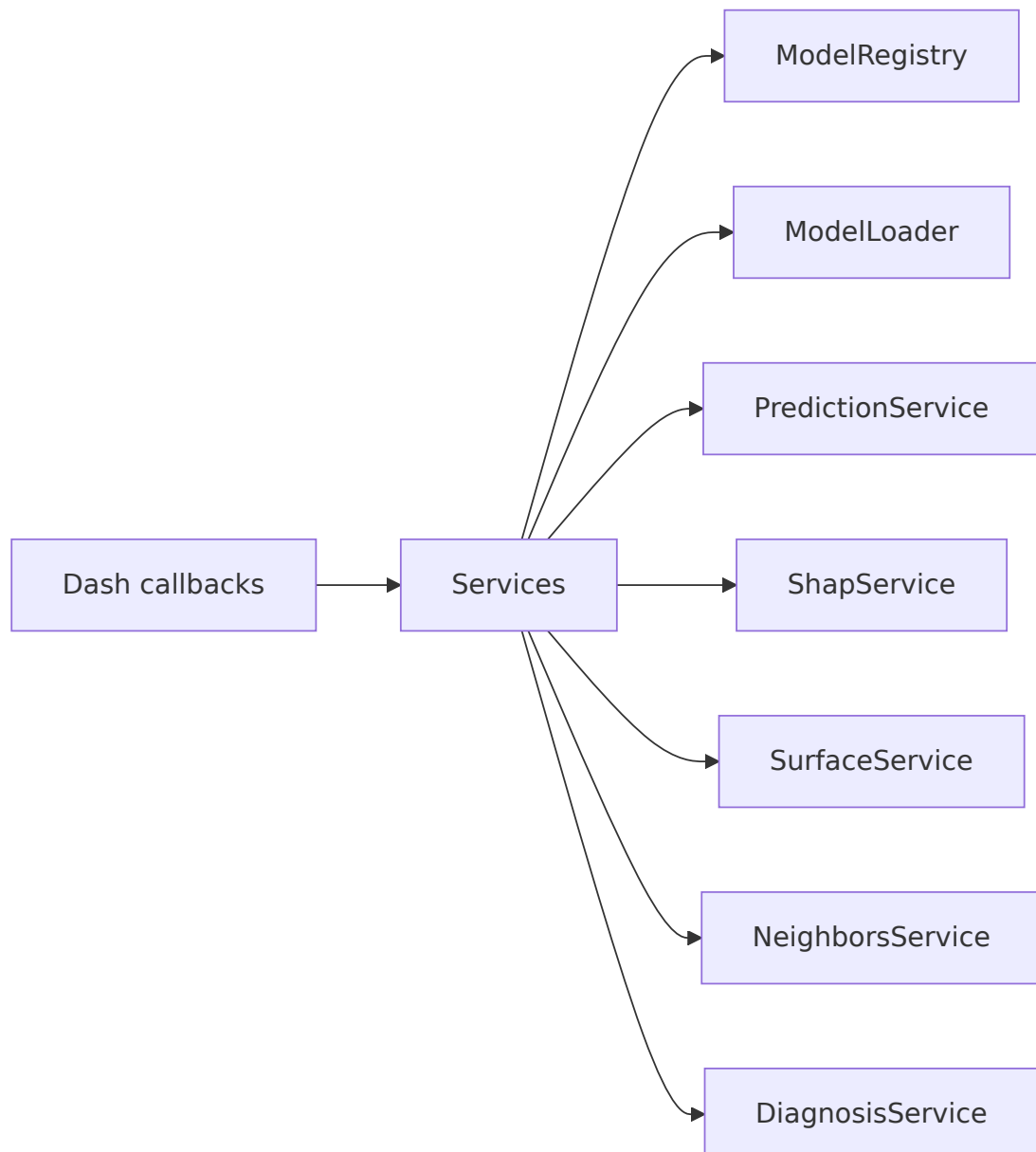


Figura 31: Arquitectura de servicios del dashboard.

Pestaña	Pregunta principal
Behaviour And Surface	¿Cómo responde el modelo ante cambios de money-ness, vencimiento y otras variables financieras?
Global Explainability	¿Qué variables y patrones dominan las predicciones globales?
Sample Explainability	¿Por qué una muestra concreta recibe esa volatilidad y tiene soporte histórico?
Diagnosis	¿Dónde acierta y dónde falla el modelo en la superficie?

Cuadro 3: Pestañas del dashboard y función analítica.

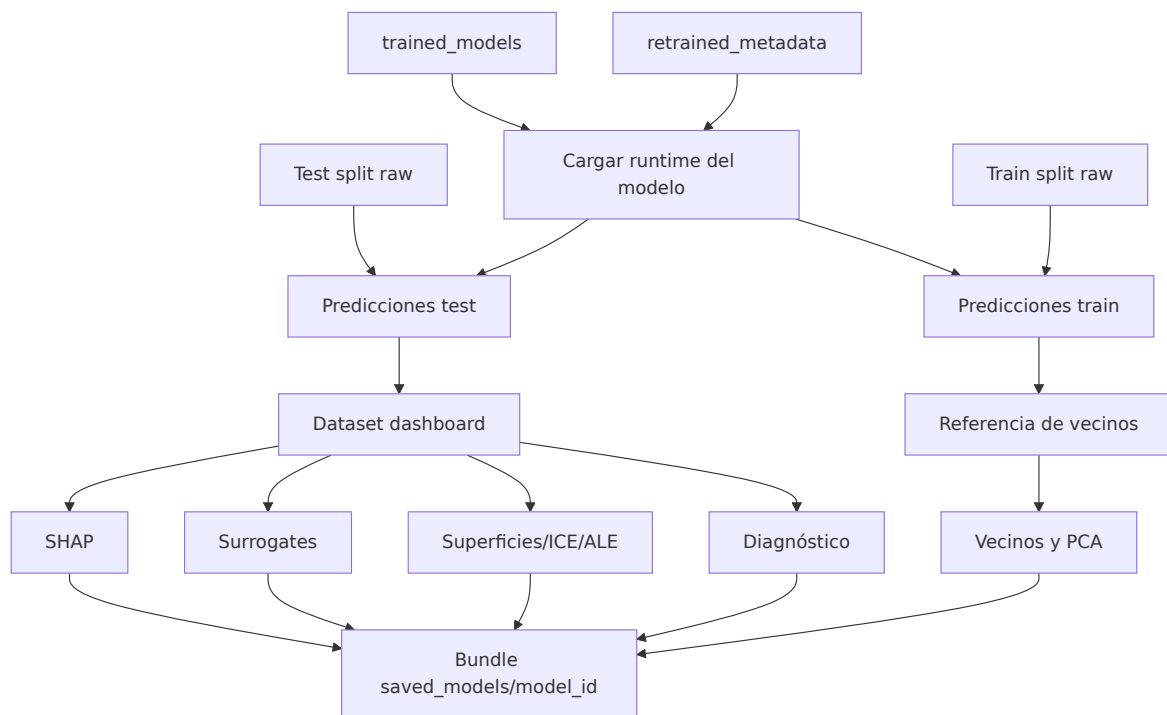


Figura 32: Conversion de modelo final a paquete visualizable.

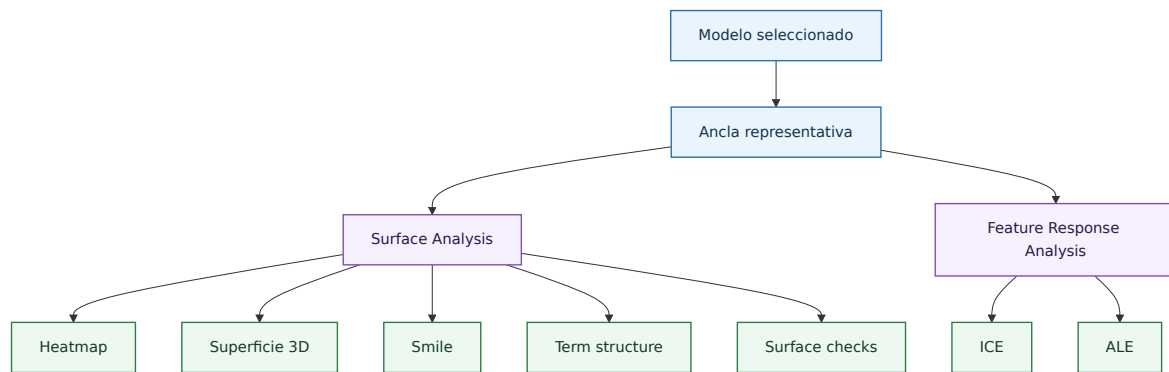


Figura 33: Flujo de la pestaña Behaviour And Surface.

La petición incluye modelo, tipo de opción, strike, subyacente, tiempo a vencimiento, tipo de interés y, opcionalmente, código de contrato y volatilidad real.

La respuesta de predicción incluye el modelo, la volatilidad estimada y la entrada normalizada. La respuesta de explicabilidad local añade waterfall SHAP, payload de contribuciones, vecinos históricos y distancias. API y dashboard comparten modelos y paquetes autocontenidos de dashboard, lo que reduce el riesgo de explicar una versión distinta de la usada para predecir.

El backend de almacenamiento puede ser local o GCP. En GCP se usan Cloud Storage para modelos y paquetes de dashboard, un registro de imágenes de contenedor, Cloud Run para API y dashboard, e IAM para permisos. Terraform define los recursos y permite reproducir la infraestructura. La parte crítica es mantener sincronizados modelo, scaler, metadatos y paquete explicativo; por eso el reentrenamiento final dispara su construcción después de guardar el modelo.

Metodológicamente, esta capa cumple tres funciones.

- **Función de servicio:** exponer predicción y explicabilidad local con el mismo contrato de entrada que utiliza la interfaz visual.
- **Función de consistencia:** asegurar que dashboard y API cargan exactamente el mismo modelo y el mismo paquete explicativo.
- **Función de despliegue:** separar imágenes, dependencias y escalado de API y dashboard para que cada servicio pueda evolucionar de forma independiente.

Esta separación se observa también en `src/api` y `src/dashboard`: la API concentra endpoints HTTP, mientras que el dashboard concentra composición visual, servicios y callbacks. Ambas piezas reutilizan almacenamiento y configuración comunes, lo que facilita auditoría y versionado.

El recorrido operativo de una consulta ayuda a entender mejor esta capa.

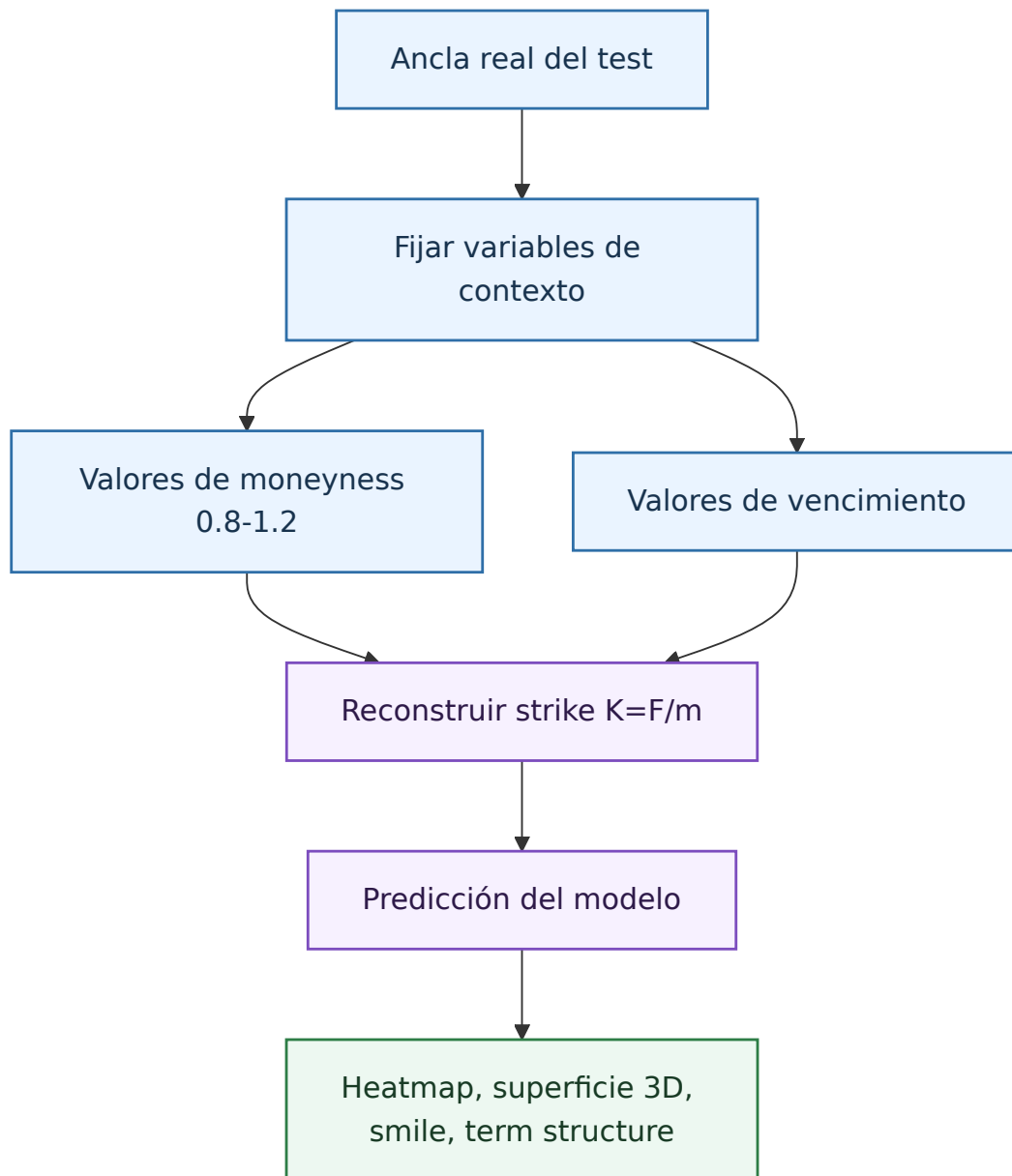


Figura 34: Construcción de superficies locales de volatilidad.

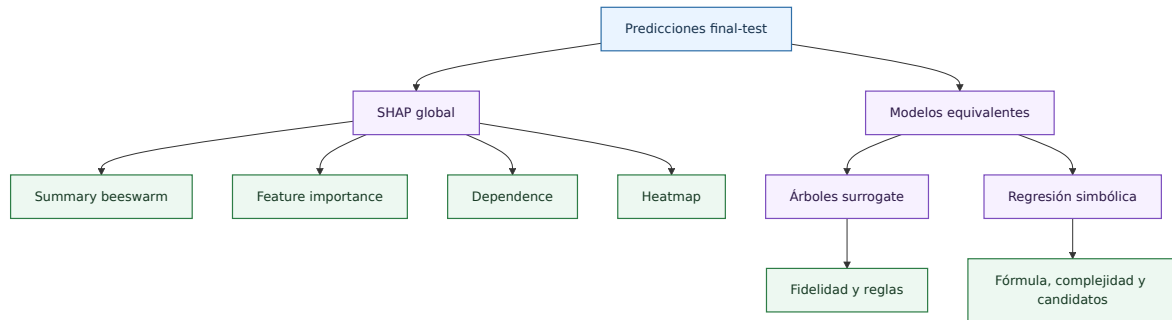


Figura 35: Flujo de la pestaña Global Explainability.

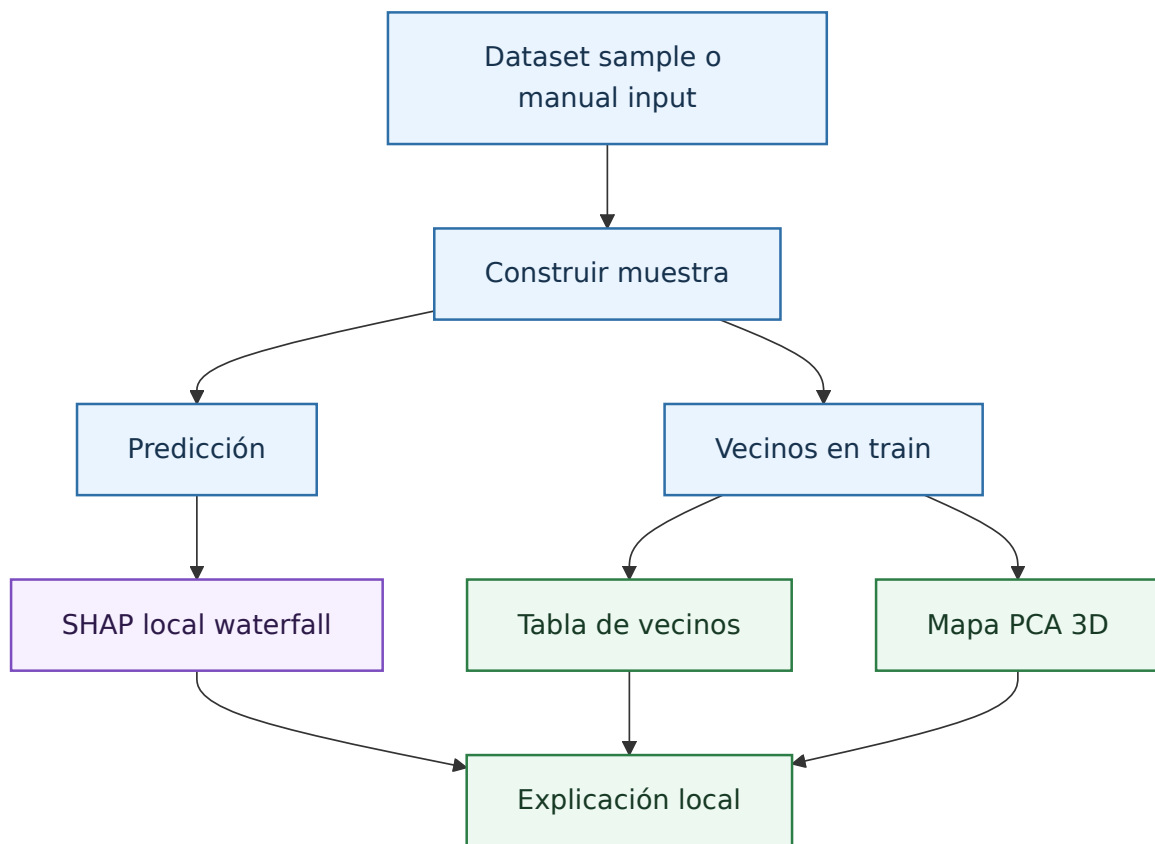


Figura 36: Flujo de la pestaña Sample Explainability.

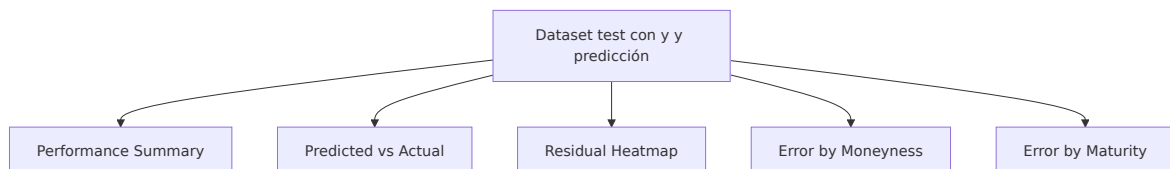


Figura 37: Vistas de diagnostico del modelo final.

1. La aplicación cliente envía variables financieras visibles y, si existe, información contextual adicional como contrato o volatilidad observada.
2. La API valida el payload, reconstruye internamente las features transformadas y ejecuta el runtime del modelo seleccionado.
3. Si se pide explicabilidad local, la misma petición desencadena cálculo de contribuciones, recuperación de vecinos y empaquetado de la respuesta interpretativa.
4. Dashboard y API leen los mismos recursos persistidos, por lo que la diferencia entre ambos canales está en la presentación y no en la lógica del modelo.

Esta última idea es especialmente importante. La memoria no presenta dos sistemas paralelos, uno visual y otro programático, sino dos interfaces sobre la misma base modelística y explicativa. Eso reduce riesgo de inconsistencia y hace más defendible el uso profesional del conjunto.

3.10 REPRODUCIBILIDAD Y TRAZABILIDAD

La reproducibilidad se apoya en cuatro mecanismos: configuración JSON, enums de esquemas, salidas persistidas y tests. La configuración declara columnas, splits, métricas, progressive training, tamaños de muestra de dashboard, parámetros SHAP, vecinos y regresión simbólica. Los enums centralizan nombres de columnas y tipos de dominio. Las salidas persistidas guardan modelo, scaler cuando aplica, metadatos y paquete de dashboard. Los tests cubren utilidades de datos, modelos, dashboard, API y configuración.

Esa trazabilidad no es abstracta; está fijada en parámetros concretos.

- En `resources/volatility_models_config.json` se definen `train_size=0.85`, `lag=15`, cinco folds temporales y las penalizaciones de CE_1 y CE_2 .
- En `resources/dashboard_models_config.json` se fija el coste de las explicaciones: `shap_background_size=24`, `shap_explain_size=48`, `neighbors_k=200` y `surface_grid_size=24`.
- En `src/model2dashboard/features.py` se centralizan las entradas raw visibles y la reconstrucción del vector transformado para que entrenamiento, dashboard y API compartan la misma lógica.
- En `src/model2dashboard/artifact_builders.py` se generan con semilla controlada las muestras SHAP, anclas de comportamiento, vecinos, superficies, ICE y ALE.

La consecuencia es clara: si cambia un schema, una ruta o una transformación, el cambio queda aislado y visible. Eso reduce el riesgo de divergencias silenciosas entre el análisis entrenado y la versión realmente servida.

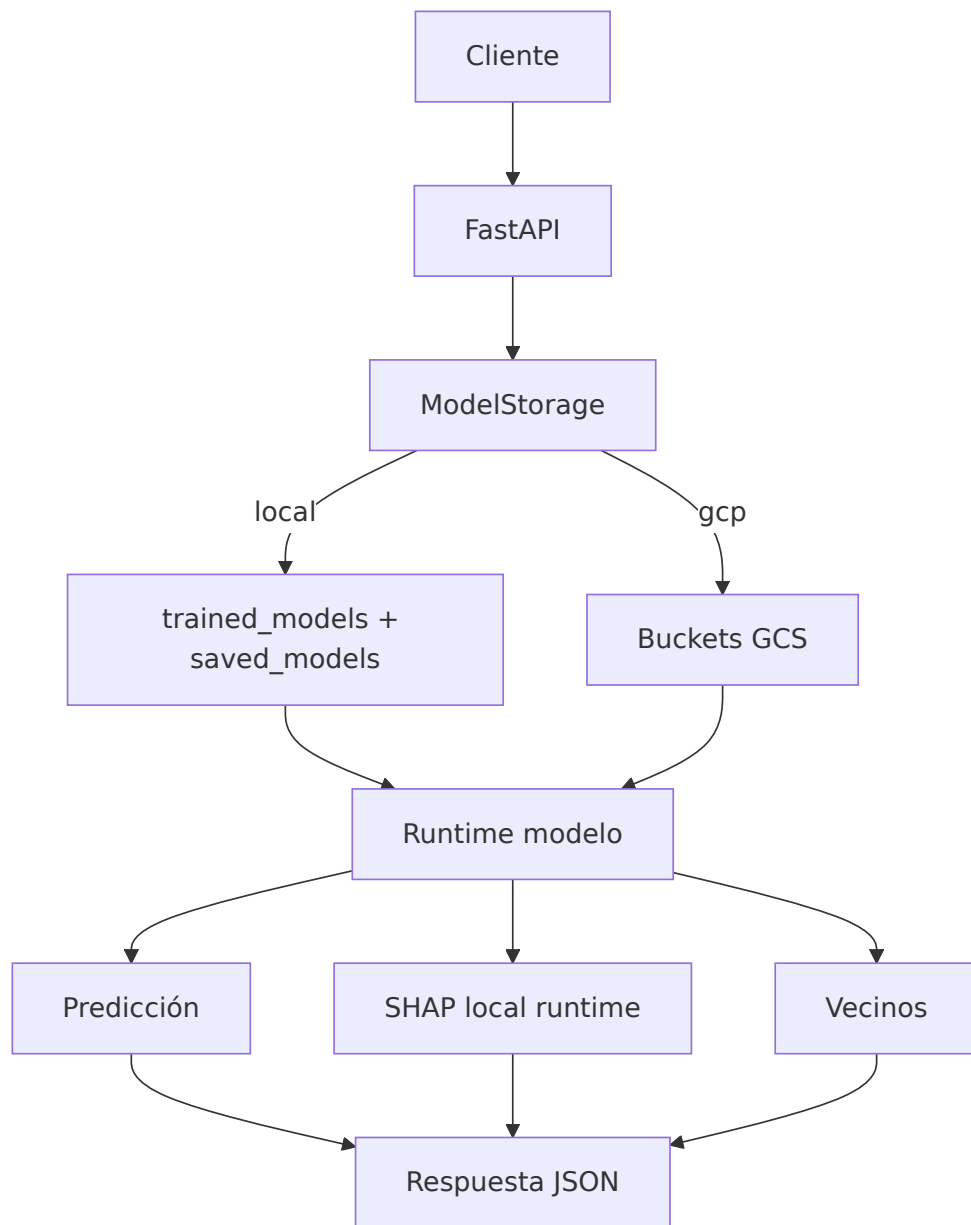


Figura 38: API, almacenamiento y runtime común.

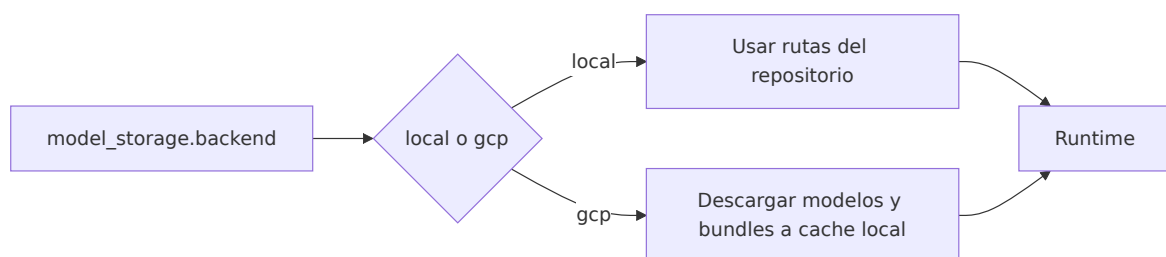


Figura 39: Selección de backend local o GCP.

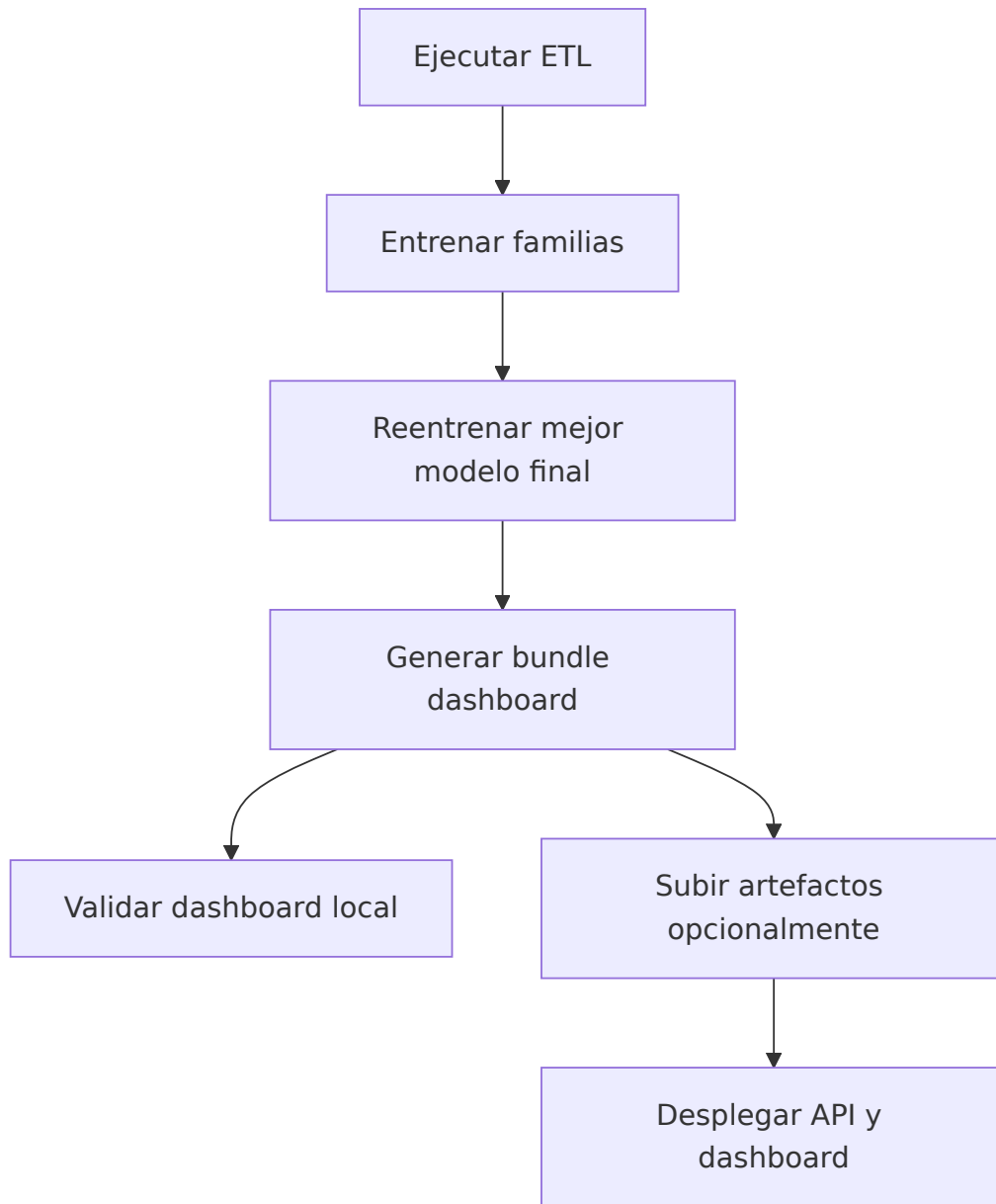


Figura 40: Flujo operativo recomendado.

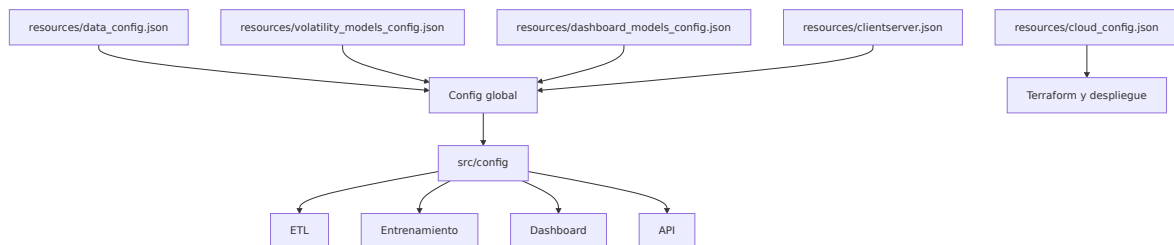


Figura 41: Configuración centralizada del proyecto.

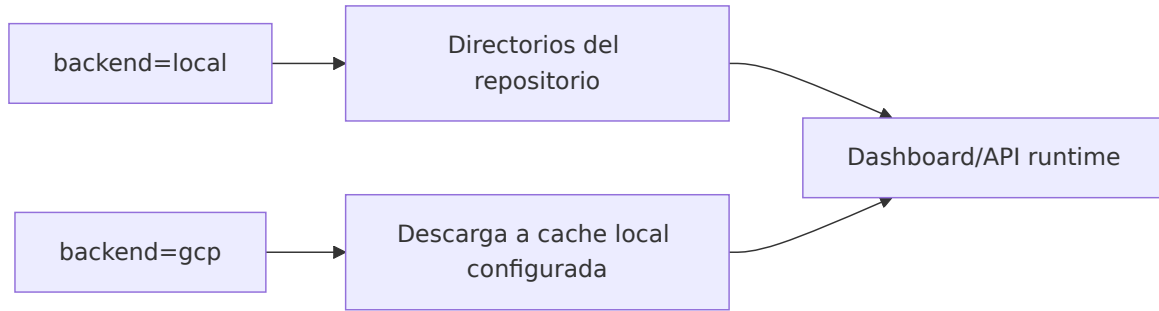


Figura 42: Carga local o GCP desde configuración cliente-servidor.

3.11 ARQUITECTURA DEL REPOSITORIO

La arquitectura del repositorio sigue una separación por responsabilidades. Esta decisión es relevante para reproducibilidad y defensa técnica: cada paquete tiene un cometido identificable y las salidas persistidas pasan de una etapa a otra sin mezclar entrenamiento, explicación y servicio.

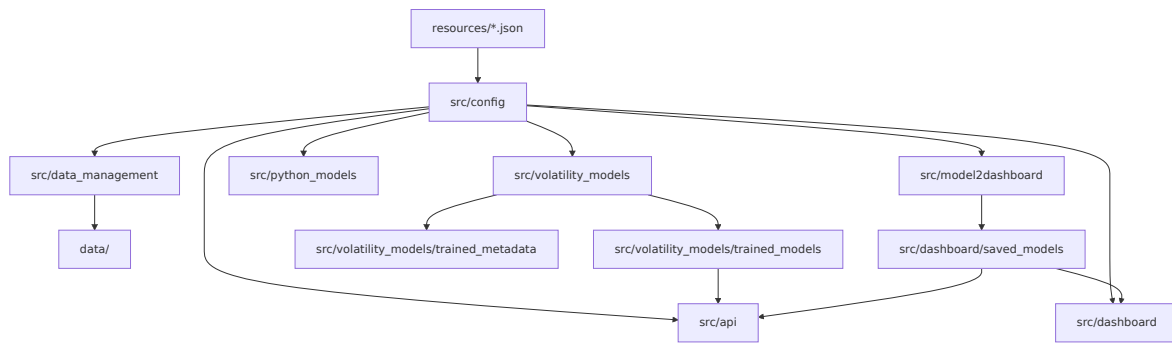


Figura 43: Estructura lógica del repositorio.

La Tabla 4 muestra que el dashboard no reentrena modelos y la API no reconstruye explicaciones desde cero salvo en casos de explicabilidad local manual. Esta separación reduce el acoplamiento y facilita detectar errores. Si una predicción de API no coincide con la vista del dashboard, el problema puede acotarse a carga del paquete autocontenido, transformación de features o versión de modelo.

La trayectoria lógica de ejecución ayuda a entender esa separación.

- `src/config` y `src/enums` fijan contratos de datos, nombres y parámetros.
- `src/data_management` produce `VOLATILITY_DB` a partir de fuentes de mercado y tipos.
- `src/volatility_models` parte esa base, genera features, selecciona familias y persiste el modelo final.

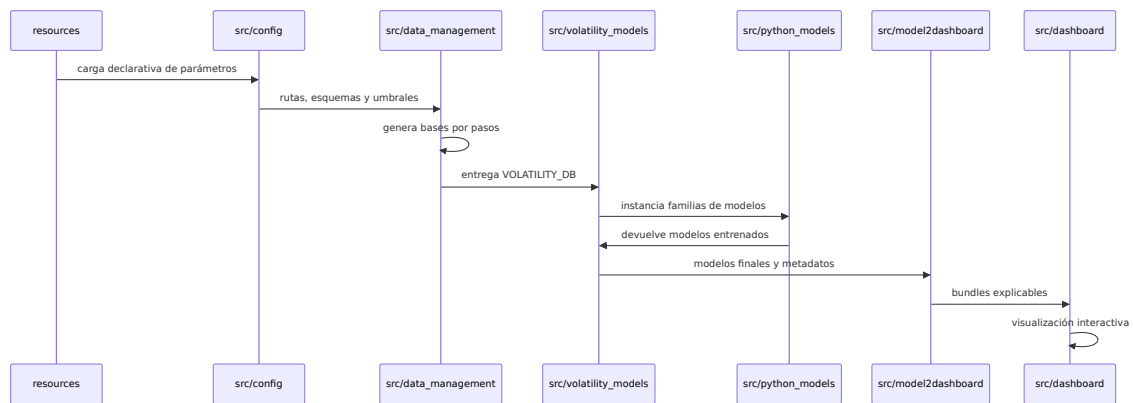


Figura 44: Secuencia de ejecución entre paquetes.



Figura 45: Cadena de salidas persistidas del proyecto.

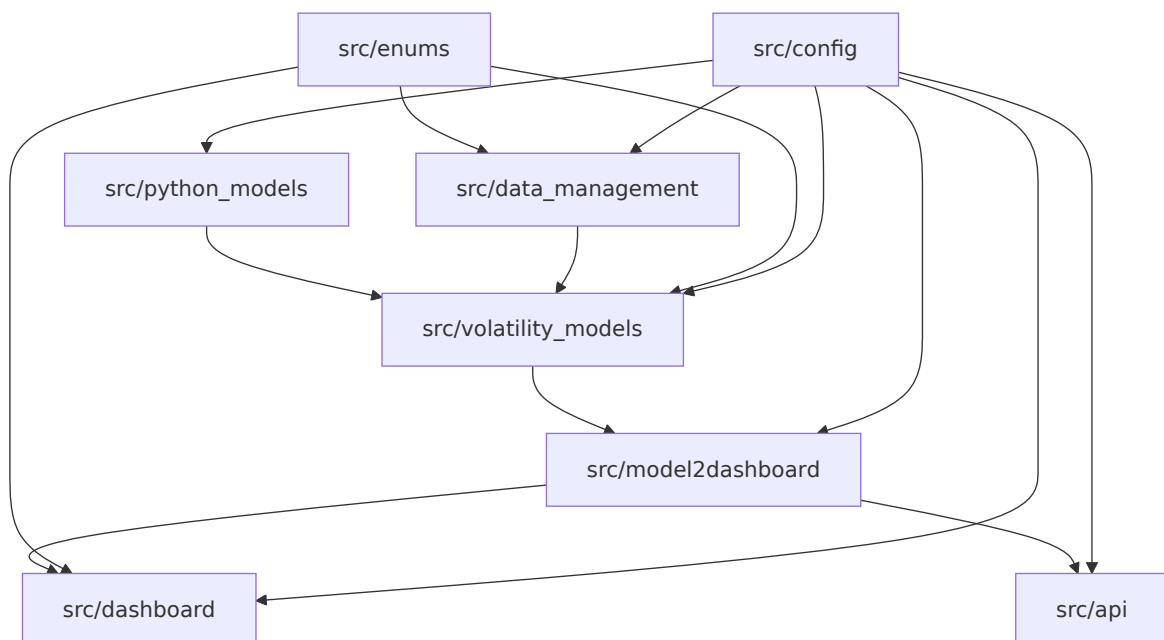


Figura 46: Dependencias principales entre paquetes de src.

Paquete o carpeta	Responsabilidad principal
src/config	Carga de configuración, rutas absolutas, logging y objetos de configuración por dominio.
src/enums	Definición centralizada de columnas, tipos de contrato, tipos de opción y nombres de modelos.
src/data_management	ETL de datos: loaders, builders, validaciones y utilidades de contrato.
src/volatility_models	Preparación de datos de entrenamiento, splits, k-folds, selección y reentrenamiento.
src/python_models	Abstracciones de familias de modelos y estructuras persistentes de dashboard.
src/model2dashboard	Conversión de modelos finales en paquetes explicativos autocontenidos.
src/dashboard	Aplicación Dash, servicios, callbacks, gráficos y utilidades de validación.
src/api	API FastAPI para predicción y explicabilidad local.
terraform	Infraestructura cloud reproducible: buckets, registro de imágenes, Cloud Run e IAM.

Cuadro 4: Arquitectura lógica del repositorio.

- src/model2dashboard toma ese modelo y genera un paquete consultable con predicciones, explicaciones y diagnósticos.
- src/dashboard y src/api consumen el mismo paquete sin alterar el predictor entrenado.

Vista así, la arquitectura no es sólo una organización de carpetas. Es una forma de aislar responsabilidades, simplificar depuración y poder justificar cada paso del flujo de principio a fin.

3.12 PLAN DE VALIDACIÓN FUNCIONAL

Además de la validación estadística, el proyecto requiere validación funcional. El sistema debe demostrar que los modelos y paquetes de dashboard se cargan, que las features se reconstruyen de forma consistente y que las explicaciones corresponden al modelo seleccionado. La validación funcional se organiza en cuatro niveles:

1. Validación de datos: esquemas, tipos, claves, precios, cantidades, vencimientos, strikes y subyacente temporalmente anterior.

2. Validación de entrenamiento: splits temporales, exclusividad de contratos, métricas por fold, selección de hiperparámetros y persistencia de metadatos.
3. Validación del paquete autocontenido: correspondencia entre modelo, scaler, predicciones, residuos, SHAP, vecinos, superficies y surrogates.
4. Validación de servicio: endpoints de salud, predicción, explicabilidad local, carga desde backend local o GCP y coherencia con dashboard.

Este plan no sustituye la revisión de resultados, pero evita que la memoria dependa únicamente de tablas finales. Un modelo con buen RMSE no sería aceptable si los componentes explicativos no correspondieran a ese mismo modelo o si la API reconstruyera las features con una transformación distinta.

La evidencia de esta validación aparece en dos formas complementarias.

- **Automática**, mediante tests sobre configuración, utilidades de datos, runtime de modelos, API, plots y servicios del dashboard.
- **Manual guiada**, mediante contraste entre vistas del dashboard, comparación API-dashboard y revisión de muestras representativas con explicabilidad local.

La primera detecta errores de contrato o serialización. La segunda detecta incoherencias funcionales que sólo aparecen al leer el modelo como superficie financiera y como sistema de consulta real.

3.13 RIESGOS METODOLÓGICOS Y CONTROLES

Los principales riesgos metodológicos se anticipan desde el diseño. El primero es el lookahead bias, controlado mediante orden temporal y subyacentes anteriores a la opción. El segundo es data snooping, mitigado separando búsqueda de hiperparámetros, reentrenamiento y evaluación final. El tercero es memorización de contratos, tratado con exclusividad de contratos entre splits. El cuarto es extrapolación en regiones poco cubiertas, diagnosticada con vecinos históricos y heatmaps de error. El quinto es confundir explicabilidad del modelo con verdad de mercado; por ello los surrogates se presentan con métricas de fidelidad y los resultados se contrastan frente a residuales y superficies.

La Tabla 5 resume esta lógica.

Leída en conjunto, la tabla resume una idea central del trabajo: cada riesgo importante tiene un control técnico explícito y una evidencia asociada en el dashboard, la API o los metadatos persistidos. Esa correspondencia entre riesgo, control y comprobación es lo que hace defendible la metodología más allá de la métrica final.

Riesgo	Control aplicado
Uso de información futura	Orden temporal, lag y subyacente anterior a la operación de opción.
Selección oportunista	K-folds temporales y métricas compuestas CE_1 y CE_2 .
Memorización de contrato	Asignación de contratos compartidos a un único split.
Extrapolación local	Vecinos históricos, distancias y mapas PCA locales.
Explicaciones engañosas	SHAP, ICE/ALE, surrogates y diagnóstico usados de forma complementaria.
Inconsistencia operativa	Paquete autocontenido con modelo, metadatos, scaler y explicaciones.

Cuadro 5: Riesgos metodológicos y controles implementados.

RESULTADOS Y DISCUSIÓN

El entrenamiento final deja un resultado principal: el mejor equilibrio fuera de muestra corresponde a Random Forest con entrenamiento estándar. En el conjunto de test obtiene $RMSE = 0,088403$, $MAE = 0,054366$ y $R^2 = 0,790941$. La lectura de resultados se organiza en tres planos: rendimiento predictivo, impacto del entrenamiento progressive ATM y capacidad de explicación del modelo final mediante dashboard.

4.1 RESULTADOS

4.1.1 Comparativa general de modelos

Modelo	Entrenamiento	MAE test	RMSE test	R^2 test	CE ₂
Regresión lineal	estándar	0.067585	0.183824	0.096059	0.133738
Regresión lineal	progressive ATM	0.070544	0.187747	0.057069	0.132802
Random Forest	estándar	0.054366	0.088403	0.790941	0.089361
Random Forest	progressive ATM	0.054337	0.088533	0.790326	0.091597
XGBoost	estándar	0.054604	0.091735	0.774885	0.086826
XGBoost	progressive ATM	0.057295	0.113606	0.654747	0.093189
Red neuronal secuencial	estándar	0.055425	0.113506	0.655354	0.090209
Red neuronal secuencial	progressive ATM	0.055634	0.120161	0.613754	0.096304
Quantum Inspired NN	estándar	0.056056	0.143025	0.452787	0.089332
Quantum Inspired NN	progressive ATM	0.055597	0.117404	0.631276	0.100256

Cuadro 6: Comparación final de modelos en test y métrica compuesta de reentrenamiento train/validation.

La Tabla 6 muestra que las familias de árboles dominan el test. Random Forest estándar obtiene el menor RMSE y el mayor R^2 ; XGBoost estándar queda cerca, pero con RMSE ligeramente superior. Las redes neuronales no superan a las familias de árboles en esta muestra, aunque la variante Quantum Inspired mejora claramente al aplicar progressive ATM frente a su versión estándar.

La comparación deja tres mensajes claros.

- El problema parece premiar modelos tabulares capaces de capturar interacciones no lineales sin requerir un ajuste tan delicado como el de las redes.
- La diferencia entre Random Forest y XGBoost es pequeña en valor absoluto, pero suficiente para inclinar la selección final hacia el bosque.
- Las arquitecturas más complejas no aportan automáticamente mejor generalización, lo que refuerza la necesidad de validar fuera de muestra y no por intuición tecnológica.

Por tanto, el resultado agregado ya sugiere una lectura metodológica útil: en esta base de volatilidad implícita, la ventaja no proviene de la familia más sofisticada en abstracto, sino de la que mejor equilibra flexibilidad, robustez y estabilidad temporal.

4.1.2 Entrenamiento estándar frente a progressive ATM

Familia	RMSE base	RMSE prog.	Δ	Lectura
Regresión lineal	0.183824	0.187747	+0.003923	Empeora levemente.
Random Forest	0.088403	0.088533	+0.000130	Prácticamente neutro.
XGBoost	0.091735	0.113606	+0.021871	Degrada test.
Red neuronal secuencial	0.113506	0.120161	+0.006655	Degrada test.
Quantum Inspired NN	0.143025	0.117404	-0.025621	Mejora clara.

Cuadro 7: Impacto agregado de progressive ATM frente a entrenamiento estándar.

El entrenamiento progressive ATM no produce una mejora universal. En Random Forest, que ya es el mejor modelo, el cambio es casi neutro; en XGBoost y en la red secuencial empeora el RMSE test; en Quantum Inspired sí actúa como regularizador útil. Esto encaja con la interpretación metodológica: ordenar por cercanía a ATM puede estabilizar arquitecturas iterativas más sensibles, pero también puede sesgar el aprendizaje si las alas contienen información necesaria para test. Por tanto, progressive ATM debe tratarse como variante experimental y no como decisión automática.

La evidencia sugiere una lectura por familias.

- En **Random Forest**, la proximidad de resultados indica que el bosque ya capta bien la señal central sin necesitar currículum de entrenamiento.
- En **XGBoost** y en la **red secuencial**, el deterioro sugiere que priorizar ATM puede dejar peor representadas zonas relevantes para la generalización final.

- En **Quantum Inspired**, la mejora apunta a que la estrategia sí puede actuar como regularizador estructural cuando la arquitectura es más sensible al orden de aprendizaje.

La conclusión práctica es sobria: progressive ATM es útil como hipótesis de trabajo y como herramienta de análisis, pero su activación debe justificarse por evidencia empírica en cada familia concreta.

4.1.3 Selección del modelo final

Elemento	Valor final
Modelo seleccionado	Random Forest
Estrategia de entrenamiento	Estándar, sin progressive ATM
Ruta del modelo	src/volatility_models/ trained_models/random_forest.joblib
Hiperparámetros principales	500 árboles, profundidad máxima 8, min_samples_leaf=10, max_features=0.5, max_samples=0.6
RMSE test	0.088403
MAE test	0.054366
R^2 test	0.790941
Argumento principal de selección	Menor RMSE test y mejor R^2 , con diferencia pequeña pero consistente frente a XGBoost.
Riesgo principal detectado	Predicciones menos dispersas que la volatilidad real: $\sigma_{\hat{y},test} = 0,149458$ frente a $\sigma_{y,test} = 0,193345$.

Cuadro 8: Ficha del modelo final seleccionado.

La decisión no se basa solo en el mínimo RMSE. Random Forest también conserva buena capacidad de generalización en train/validation: $RMSE_{train} = 0,071027$, $RMSE_{val} = 0,071080$ y $R^2_{val} = 0,653348$. XGBoost tiene el CE_2 más bajo en train/validation, pero su test queda por detrás. La elección final prioriza rendimiento fuera de muestra, estabilidad razonable y menor degradación al pasar de validación a test.

La selección final combina criterios cuantitativos y operativos.

- **Criterio cuantitativo principal:** mejor RMSE test entre todas las alternativas evaluadas.
- **Criterio de estabilidad:** gap contenido entre entrenamiento, validación y test, sin señales fuertes de sobreajuste.

- **Criterio operativo:** buen encaje con las herramientas de diagnóstico, explicabilidad y servicio generadas para el paquete final.

Con ello, la elección deja de ser una simple foto de leaderboard. El Random Forest final es el candidato que mejor soporta la transición desde el entrenamiento hasta el uso interpretado en dashboard y API.

4.1.4 Diagnóstico de rendimiento

El diagnóstico debe interpretarse a partir del test. La desviación estándar de la volatilidad real es 0.193345 y la de las predicciones de Random Forest es 0.149458. Esta diferencia indica compresión parcial de la variabilidad: el modelo captura la estructura general, pero tiende a suavizar extremos. La desviación estándar residual es 0.085859, coherente con un RMSE de 0.088403.

Las vistas del dashboard son necesarias para localizar dónde aparece esa compresión. El scatter *predicted vs actual* debe confirmar si los extremos altos y bajos se acercan a la media; el residual heatmap debe separar error por moneyness y vencimiento; y las cajas *Error by Moneyness* y *Error by Maturity* deben indicar si las alas o vencimientos cortos concentran el fallo. La conclusión accionable es que el modelo es competitivo agregadamente, pero su uso debe acompañarse de diagnóstico regional cuando se consulten contratos poco líquidos o alejados de ATM.

En la práctica, el diagnóstico debe responder tres preguntas consecutivas.

- Si el error está repartido de forma homogénea o se concentra en zonas concretas de la superficie.
- Si la dispersión de predicciones es suficiente o si el modelo tiende a comprimir sistemáticamente los extremos.
- Si los fallos observados son compatibles con escasez de datos, con dificultades intrínsecas del problema o con una arquitectura demasiado rígida.

Este nivel de detalle es necesario porque dos modelos con RMSE parecido pueden implicar riesgos operativos muy distintos. Uno puede fallar de forma estable y moderada; otro puede acertar el promedio pero perder control justo en las regiones más delicadas para valoración o revisión manual.

4.1.5 Behaviour And Surface

El modelo final debe leerse como función financiera, no solo como tabla de errores. Las superficies generadas alrededor de anclas del test permiten inspeccionar $\hat{\sigma}(m, T)$, smiles y estructuras temporales. Dado que Random Forest es un modelo por particio-

nes, la revisión visual debe prestar especial atención a escalones locales: un bosque puede aproximar bien puntos observados y aun así producir transiciones menos suaves que un modelo paramétrico.

La lectura recomendada es usar al menos tres anclas: una observación ATM líquida, una observación en alas y una observación de vencimiento corto. Si el dashboard muestra saltos alrededor de un punto, la explicación debe cruzarse con vecinos y residual heatmap antes de atribuirlo a señal financiera. Si el smile es suave y el error local bajo, la predicción es más defendible.

Una secuencia de lectura útil en esta pestaña es la siguiente:

- fijar un ancla representativa y revisar primero el heatmap local;
- comprobar después la superficie 3D para detectar escalones o zonas artificialmente planas;
- descender a smile y term structure para ver si la geometría sigue siendo razonable en cortes financieros conocidos;
- contrastar finalmente con ICE, ALE, vecinos y diagnóstico regional cuando aparezcan formas dudosas.

Este orden es importante porque evita interpretar una gráfica aislada fuera de contexto. Un escalón puede ser inocuo si coincide con una zona bien soportada y de bajo error, o problemático si aparece en una región escasa y con extrapolación local.

4.1.6 Explicabilidad global

La explicación global debe comprobar que los drivers principales son coherentes con la geometría de volatilidad: moneyness, vencimiento, subyacente, strike, tipo y tipo de opción. SHAP se calcula sobre variables raw visibles y el runtime reconstruye las features transformadas, por lo que una contribución de StrikePrice o UnderlyingPrice debe interpretarse en relación con moneyness y no como efecto causal aislado.

La lectura correcta combina ranking, signo local y dependence plots. Si una variable domina el ranking pero tiene colores mezclados o contribuciones con mucha dispersión, el modelo está usando interacciones. Esa conclusión es razonable en opciones, porque strike, subyacente y vencimiento no son independientes. Si aparecen saltos en dependence plots, deben contrastarse con las superficies y con diagnóstico por regiones.

Para que la lectura global sea sólida, conviene exigir tres coherencias a la vez.

- **Coherencia estadística:** que las variables más importantes aparezcan de forma estable en varias visualizaciones SHAP.

- **Coherencia financiera:** que los drivers dominantes sean compatibles con la geometría de moneyness, vencimiento y tipo.
- **Coherencia con error:** que un patrón llamativo en SHAP pueda contrastarse con residual heatmap, superficies y casos locales.

Si una variable parece importante pero esa importancia no se refleja ni en las superficies ni en la lectura regional del error, la conclusión debe tomarse con cautela. La explicabilidad global no sustituye al resto de vistas; las organiza.

4.1.7 Modelos equivalentes: árboles y regresión simbólica

Los modelos equivalentes explican el predictor, no la verdad de mercado. Para Random Forest, las métricas de fidelidad de árboles surrogate son:

Profundidad	RMSE fidelidad	MAE fidelidad	R^2 fidelidad
2	0.139477	0.053488	0.129099
4	0.135217	0.050783	0.181486
8	0.141899	0.058392	0.098593
16	0.144616	0.053187	0.063747

Cuadro 9: Fidelidad de árboles surrogate frente al Random Forest seleccionado.

La fidelidad de los árboles es limitada: ni siquiera profundidades altas reproducen bien el comportamiento completo del bosque. Esto no invalida el modelo principal, pero sí limita el uso de reglas simples como explicación global. La regresión simbólica generada para este paquete muestra métricas no utilizables para defensa del Random Forest, con RMSE de fidelidad extremadamente alto; por tanto, la memoria debe apoyarse en SHAP, ICE/ALE, superficies y vecinos para explicar el modelo final. La conclusión no debe forzarse: el predictor es bueno, pero no se resume bien en una fórmula compacta en esta ejecución.

La lectura correcta de estos modelos equivalentes es prudente.

- Si la fidelidad es baja, las reglas o fórmulas sólo valen como resumen parcial y no como explicación sustitutiva del modelo principal.
- Si la fidelidad mejora con mucha complejidad, la conclusión útil no es que exista una explicación compacta, sino precisamente que el predictor no se deja comprimir bien.
- Si la fidelidad fuese alta con baja complejidad, entonces sí tendría sentido dar más peso a reglas o ecuaciones en la discusión global.

4.1.8 Explicabilidad local y soporte histórico

El análisis local debe seleccionar una muestra representativa y una muestra problemática. Para cada una, el waterfall SHAP explica la predicción como suma de valor base y contribuciones. Los vecinos históricos, calculados contra train en el espacio transformado y estandarizado, indican si la muestra está dentro de una región conocida. En el paquete se guardan hasta 200 vecinos por muestra, usando una referencia de entrenamiento de hasta 20.000 observaciones.

La interpretación profesional debe combinar ambas piezas. Si una muestra tiene waterfall coherente y vecinos cercanos con volatilidades similares, la explicación es sólida. Si el waterfall parece razonable pero los vecinos están lejos, el modelo puede estar extrapolando; en ese caso, la predicción debe revisarse con diagnóstico regional y superficie local.

Como regla práctica, una explicación local es más defendible cuando se cumplen tres condiciones.

- El signo y magnitud de las contribuciones SHAP son compatibles con la intuición financiera del caso.
- Los vecinos cercanos muestran contratos, moneyness y vencimientos realmente parecidos.
- El error regional del modelo en esa zona no es anómalamente alto.

Cuando falla alguna de esas tres condiciones, la lectura debe cambiar: la explicación sigue diciendo cómo razona el modelo, pero deja de ser una justificación sólida de que la predicción sea fiable en términos de mercado.

4.2 DISCUSIÓN

La pregunta planteada en [section 1.1](#) puede responderse afirmativamente con matices. Sí es posible construir un modelo útil y consultable para volatilidad implícita de opciones del IBEX; el Random Forest seleccionado ofrece buen error fuera de muestra. El matiz es que la explicación no puede descansar en un único resumen global: la fidelidad de modelos equivalentes simples es limitada, y la defensa técnica debe combinar SHAP, superficies, ICE/ALE, vecinos y diagnóstico.

4.2.1 Lectura financiera

Financieramente, el proyecto trata la volatilidad como superficie y no como serie unidimensional. El buen RMSE agregado es relevante, pero el dato más importante

para gestión de riesgos es dónde se produce el error. La compresión de volatilidades extremas sugiere que deben revisarse alas y vencimientos cortos con más cuidado. Este comportamiento es plausible en derivados: esas regiones suelen tener menos liquidez, más ruido de microestructura y mayor sensibilidad al desfase entre opción y futuro subyacente.

- El modelo final es útil para lectura agregada y consulta, pero no debe tratarse como sustituto ciego del juicio financiero en regiones extremas.
- La geometría de la superficie sigue siendo el criterio principal de plausibilidad: smiles, term structures y suavidad local importan tanto como el error medio.
- La lectura de vecinos y soporte histórico es especialmente valiosa cuando se analiza un contrato poco líquido o lejano de ATM.

En otras palabras, el trabajo refuerza una idea clásica en derivados: el resultado útil no es un único número, sino una predicción situada dentro de una estructura de superficie, de un contexto de mercado y de una evidencia histórica comparable.

También conviene subrayar una implicación práctica. Cuando el modelo se usa como apoyo a valoración o revisión manual, la pregunta correcta no es sólo si “acierta de media”, sino si conserva lógica financiera en la zona concreta que se está consultando. Esa exigencia explica por qué el documento dedica espacio a smiles, alas, vencimientos cortos y soporte local, y no sólo a tablas globales de error.

4.2.2 *Lectura de IA y Data Science*

Desde la perspectiva de IA, la comparación muestra que modelos tabulares de árboles son más adecuados que redes densas para esta muestra. Random Forest y XGBoost aprovechan interacciones no lineales sin exigir escalado y ofrecen mejor test. La métrica CE_2 favorece a XGBoost en train/validation, pero el test favorece a Random Forest; esto ilustra por qué la selección final no debe quedarse en validación. Progressive ATM mejora Quantum Inspired, pero no a las familias que ya generalizan mejor.

- La validación temporal es más importante que una mejora marginal en entrenamiento.
- La estabilidad entre train, validation y test pesa más que una única métrica brillante en una fase intermedia.
- La explicabilidad posterior también importa: un modelo competitivo pero imposible de leer exige más apoyos y más cautela en su defensa.

Desde esta perspectiva, el trabajo también deja una lección de alcance más general: en datos financieros tabulares, una arquitectura más compleja no merece preferencia por defecto. Primero debe demostrar que generaliza mejor; después, que esa mejora compensa el coste extra de explicación, mantenimiento y servicio.

4.2.3 *Lectura de ingeniería y cloud*

La dimensión cloud aporta valor porque no se usa como adorno: resuelve almacenamiento, carga de modelos, despliegue independiente de API y dashboard y trazabilidad. La separación entre modelo, paquete explicativo, API, dashboard y almacenamiento reduce acoplamientos y facilita auditoría. La principal cautela operativa es mantener sincronizados modelo, scaler, metadatos y componentes explicativos; de lo contrario, el sistema podría predecir con una versión y explicar con otra.

- El almacenamiento centralizado evita duplicidades manuales y facilita compartir la misma versión entre servicios.
- El despliegue desacoplado permite que un problema en dashboard no bloquee necesariamente la API, y viceversa.
- La reproducibilidad de infraestructura es útil sólo si el pipeline mantiene coherencia entre modelo servido y explicaciones persistidas.

La consecuencia práctica es que la ingeniería no aparece aquí como una capa secundaria. Es parte del argumento de calidad del trabajo: un modelo que no puede versionarse, servirse y auditarse con coherencia pierde valor aunque su ajuste estadístico sea aceptable.

4.2.4 *Limitaciones*

Las limitaciones se agrupan en cuatro bloques. Primero, la volatilidad implícita depende de la calidad del enlace opción-subyacente y del solver Black-76. Segundo, el conjunto de datos es heterogéneo en liquidez y cobertura de moneyness/vencimiento, por lo que los extremos deben diagnosticarse localmente. Tercero, la explicabilidad aproxima comportamientos del modelo y no sustituye validación económica. Cuarto, el despliegue cloud exige disciplina de versionado para evitar inconsistencias entre modelos, paquetes de dashboard y metadatos.

- No toda región del espacio de entrada tiene la misma densidad histórica ni la misma calidad de soporte.
- Un modelo explicable sigue pudiendo estar equivocado si aprende sesgos sistemáticos de los datos.
- Las visualizaciones ayudan a detectar problemas, pero no convierten por sí solas una predicción en económicamente válida.

Además, hay una limitación estructural que conviene dejar explícita: el proyecto explica muy bien cómo razona el modelo entrenado, pero no agota por sí solo la validación económica del mercado. Siempre queda margen para contrastar con enfoques paramétricos, con criterios de liquidez más finos o con reglas de suavidad impuestas ex ante sobre la superficie.

CONCLUSIONES

Este TFM desarrolla una cadena integral para estimar volatilidad implícita en opciones del IBEX y convertir el resultado en un sistema explicable y operable. El trabajo parte de datos de mercado, construye una base de volatilidad mediante inversión de Black-76, entrena familias de modelos bajo validación temporal y genera explicaciones y diagnósticos. El modelo final seleccionado es Random Forest estándar, con $RMSE = 0,088403$, $MAE = 0,054366$ y $R^2 = 0,790941$ en test.

La primera conclusión es que la calidad de un modelo de volatilidad no puede evaluarse solo con métricas agregadas. En derivados, la forma de la superficie, el comportamiento por moneyness y vencimiento y la localización del error son elementos centrales. Por ello el dashboard no se plantea como una capa estética, sino como parte del mecanismo de validación. En el modelo seleccionado aparece comprensión parcial de variabilidad, ya que la desviación estándar de predicciones en test es inferior a la de la volatilidad real.

La segunda conclusión es que la explicabilidad aporta valor en dos planos. En ingeniería, ayuda a detectar fallos, fugas, sesgos y discontinuidades. En finanzas, permite justificar predicciones y analizar si el modelo se comporta de forma compatible con intuición económica y revisión profesional.

La tercera conclusión es que la arquitectura de software condiciona la utilidad del modelo. Al separar ETL, entrenamiento, paquetes explicativos, API, dashboard y almacenamiento, el proyecto facilita reproducibilidad, mantenimiento y despliegue. Esta separación también reduce el riesgo de que las explicaciones se calculen sobre versiones distintas de las usadas en predicción.

La cuarta conclusión es que progressive ATM no debe presentarse como mejora universal. En esta ejecución mejora la arquitectura Quantum Inspired, pero no al Random Forest final ni a XGBoost. Su utilidad depende de si estabiliza la zona ATM sin degradar alas ni vencimientos menos representados.

Como líneas futuras, se proponen: ampliar el análisis con más ventanas temporales, incorporar métricas específicas por liquidez, explorar modelos con restricciones de

suavidad financiera, automatizar validaciones de superficie más exigentes y reforzar el versionado de modelos y paquetes explicativos en cloud. También sería natural comparar la superficie aprendida con enfoques paramétricos clásicos o con modelos híbridos que impongan estructura financiera explícita.

FUNDAMENTOS DE SHAP

SHAP se basa en una lectura cooperativa de la contribución de variables a una predicción. Su ventaja práctica es que proporciona una descomposición aditiva como la de (9). En este proyecto se usa con un explicador de permutación para tratar de forma homogénea modelos heterogéneos. Esta decisión sacrifica eficiencia frente a explicadores especializados, pero aporta una semántica común entre familias.

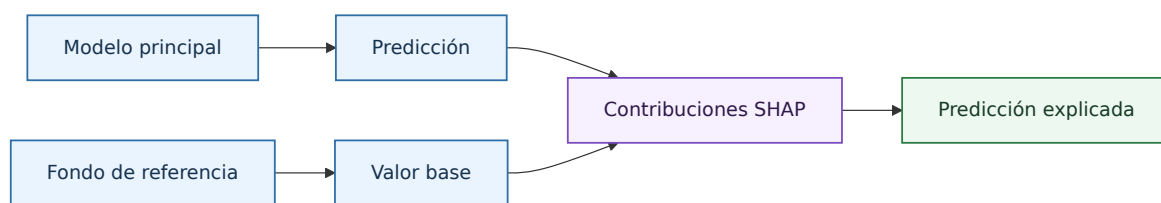


Figura 47: Descomposicion SHAP desde valor base hasta predicción explicada.

La atribución SHAP procede de los valores de Shapley. Para una feature j , el valor se interpreta como la contribución marginal media de esa feature al incorporarse a todos los subconjuntos posibles de variables:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)], \quad (15)$$

donde N es el conjunto de features, S un subconjunto que no contiene j , p el número de features, $v(S)$ el valor esperado del modelo cuando se conocen las variables de S , y ϕ_j la contribución asignada a j .

La interpretación de SHAP debe ser cuidadosa. Un valor SHAP positivo indica que, para una observación y respecto al valor base, la variable empuja la predicción al alza. No implica causalidad. Además, si las variables están correlacionadas, la atribución depende de cómo el explicador define la ausencia o permutación de features.

El repositorio usa `shap.Explainer(..., algorithm="permutation")`. El presupuesto mínimo de evaluación por fila es:

$$max_evals = 2p + 1. \quad (16)$$

El fondo se toma de una muestra controlada por `shap_background_size`; las filas explicadas globalmente por `shap_explain_size`; y las filas locales por `sample_option_size`. Las explicaciones se calculan sobre variables raw visibles: `OptionType`, `StrikePrice`, `UnderlyingPrice`, `TimeToExpiration` y `Rate`. Esta decisión mejora la lectura financiera, aunque internamente el runtime reconstruya `logMoneyness`, `sqrtTTEYears` y el resto de features derivadas mediante el encoder de explicabilidad.

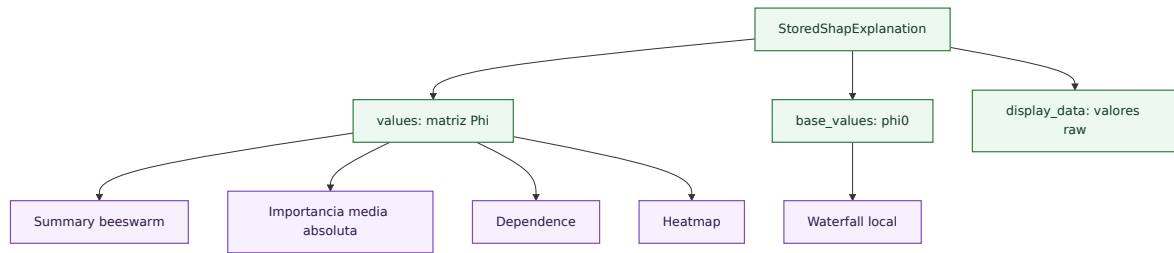


Figura 48: Estructura de datos usada por las visualizaciones SHAP.

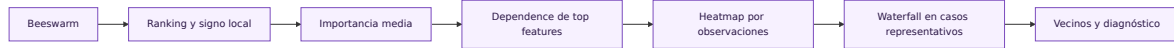


Figura 49: Lectura conjunta de visualizaciones SHAP.

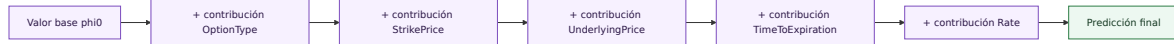


Figura 50: Construcción de una explicación local waterfall.

El beeswarm muestra contribuciones individuales por feature; la importancia media absoluta ordena variables por $\frac{1}{n} \sum_i |\phi_j(x_i)|$; el dependence plot relaciona x_{ij} con $\phi_j(x_i)$; y el heatmap permite comparar perfiles explicativos entre observaciones. El waterfall local parte de ϕ_0 y suma contribuciones hasta $\hat{\sigma}(x_i)$. Los vecinos históricos completan la explicación local: SHAP identifica drivers de predicción y los vecinos indican soporte empírico.

Una lectura disciplinada de SHAP en este proyecto sigue cuatro pasos.

- **Paso 1:** revisar el ranking global para identificar qué variables organiza el modelo con más intensidad.
- **Paso 2:** contrastar esas variables con dependence plots para ver si la relación es monótona, heterogénea o claramente interactiva.
- **Paso 3:** bajar a waterfalls locales cuando interese justificar un contrato concreto.

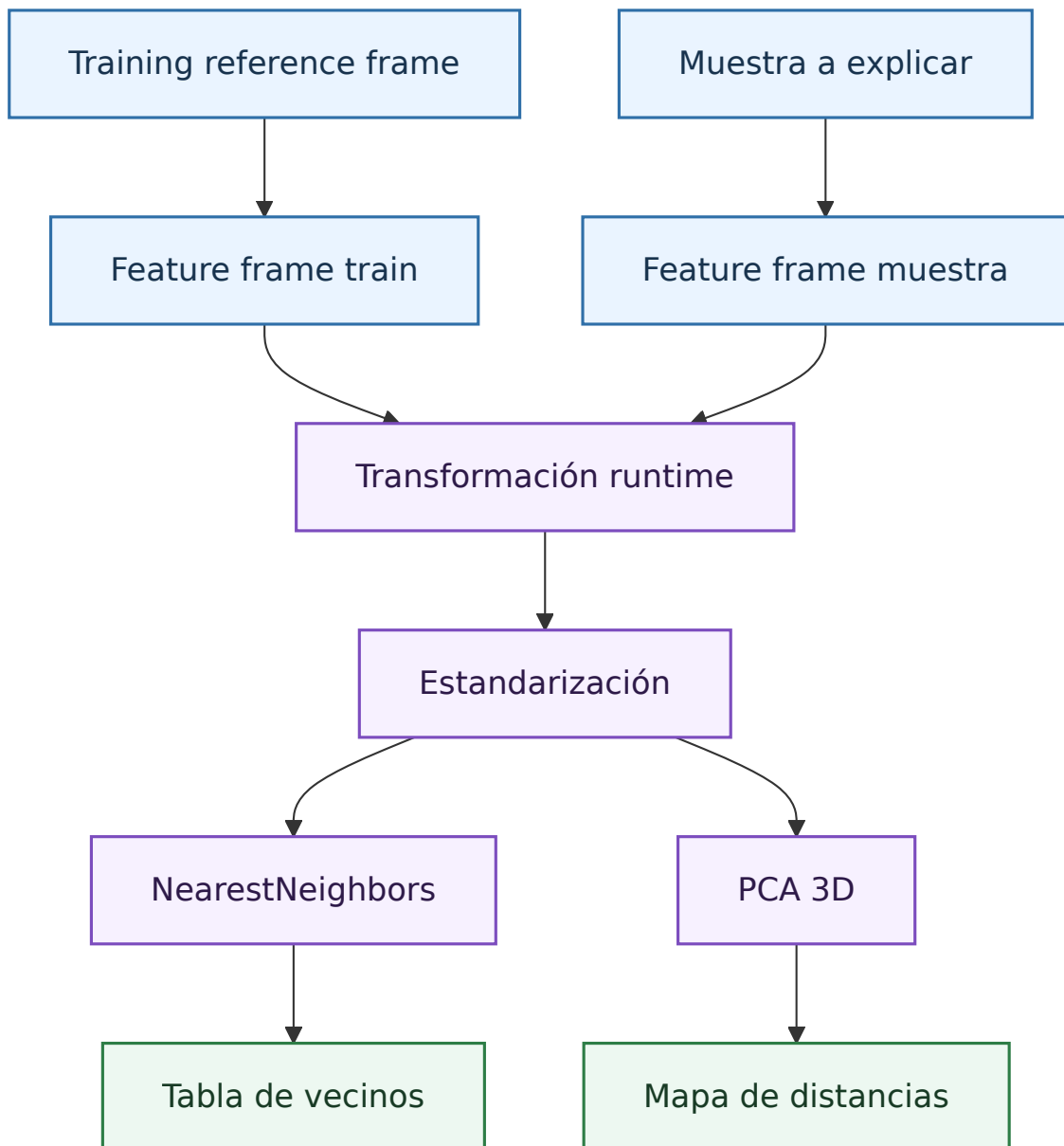


Figura 51: Calculo de vecinos y proyeccion local.

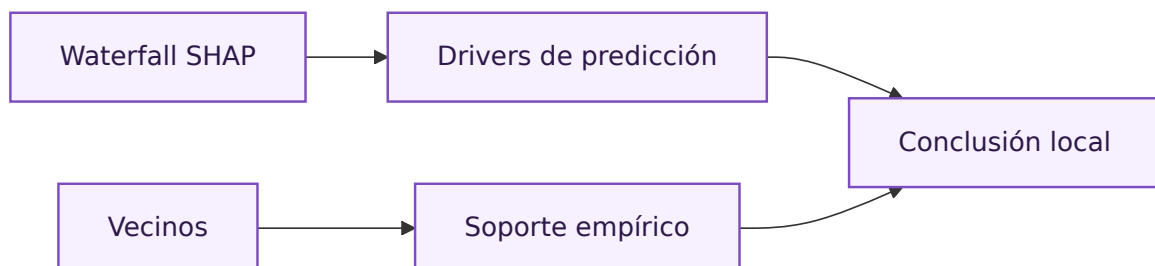


Figura 52: Relacion entre SHAP local y soporte historico.

- **Paso 4:** validar el contexto con vecinos y diagnóstico regional, especialmente si la muestra está en alas o en vencimientos cortos.

Este anexo es importante porque evita dos errores frecuentes. El primero es leer SHAP como causalidad económica. El segundo es creer que una explicación local muy detallada equivale a una garantía de calidad de la predicción. SHAP explica cómo reparte el modelo la predicción; no demuestra que la relación aprendida sea una ley financiera.

CURVAS ICE

ICE, *Individual Conditional Expectation*, muestra cómo cambia la predicción para observaciones individuales cuando se modifica una feature y se mantienen las demás constantes. Para una feature j , una observación x_i y un valor contrafactual z :

$$ICE_i^{(j)}(z) = f(z, x_{i,-j}), \quad (17)$$

donde $x_{i,-j}$ representa todas las variables salvo la feature analizada. Cada línea ICE es una trayectoria individual de predicción.

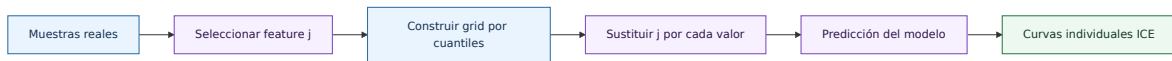


Figura 53: Construcción de curvas ICE por feature.

En el proyecto, `build_ice_frame` muestrea hasta `ice_sample_size=24` filas, crea un grid de `curve_points=25` por cuantiles y evalúa `StrikePrice`, `UnderlyingPrice`, `TimeToExpiration` y `Rate`. ICE revela heterogeneidad: curvas paralelas indican efectos similares; pendientes diferentes o cruces sugieren interacciones; saltos o dientes pueden indicar discontinuidades del modelo o zonas poco soportadas.

ICE no debe confundirse con PDP. Un PDP promedia curvas ICE:

$$PDP_j(z) = \frac{1}{n} \sum_{i=1}^n f(z, x_{i,-j}). \quad (18)$$

El promedio puede ocultar comportamientos opuestos entre observaciones. Además, ICE puede generar contrafactuales poco realistas, por ejemplo cambiar strike manteniendo fijo el resto hasta producir moneyness fuera de distribución. Por eso sus curvas deben leerse con vecinos, superficie local y diagnóstico regional.

En la práctica, conviene usar ICE para responder preguntas muy concretas.

- Qué ocurre con la predicción si se mueve el strike manteniendo fijo el contexto de una observación real.
- Si el efecto del vencimiento es homogéneo o depende mucho del régimen de moneyness.
- Si el tipo de interés tiene un impacto directo o aparece diluido frente a las variables geométricas de la superficie.

Patrones de lectura útiles:

- **Curvas casi paralelas:** el modelo usa la feature con una respuesta relativamente estable.
- **Curvas que se cruzan:** la relación depende claramente del contexto de otras variables.
- **Dientes o escalones:** posible efecto de particiones del modelo, sparsidad local o regiones poco observadas.

Por tanto, ICE no se usa aquí como una figura decorativa. Se usa para revelar heterogeneidad que un promedio global escondería y para someter al modelo a una lectura funcional cercana a la intuición financiera del usuario.

CURVAS ALE

ALE, *Accumulated Local Effects*, estima el efecto medio acumulado de una feature usando variaciones locales dentro de regiones observadas. Complementa a ICE porque resume un efecto agregado menos vulnerable a combinaciones contrafactuales irreales cuando las variables están correlacionadas.

$$ALE_j(z) = \int_{z_0}^z E \left[\frac{\partial f(X)}{\partial X_j} \mid X_j = s \right] ds - C. \quad (19)$$

La constante C centra la curva para que su media sea cero. Un valor ALE positivo no significa volatilidad positiva, sino efecto acumulado por encima del promedio de referencia.

La implementación por bins calcula bordes por cuantiles entre 0.05 y 0.95, con 13 puntos. Para cada intervalo $[l_k, u_k]$, selecciona observaciones I_k , predice dos escenarios locales y calcula:

$$\Delta_k = \frac{1}{|I_k|} \sum_{i \in I_k} [f(u_k, x_{i,-j}) - f(l_k, x_{i,-j})]. \quad (20)$$

Después acumula y centra:

$$ALE_k^u = \sum_{r \leq k} \Delta_r, \quad ALE_k = ALE_k^u - \frac{1}{K} \sum_{r=1}^K ALE_r^u. \quad (21)$$

En opciones, ALE es útil porque `StrikePrice` y `UnderlyingPrice` determinan moneyness, `TimeToExpiration` depende de calendarios de vencimiento y `Rate` depende de fecha y plazo. ALE no elimina todas las dificultades de correlación, pero evita promediar de forma indiscriminada sobre combinaciones globales poco plausibles [Apley and Zhu, 2020].

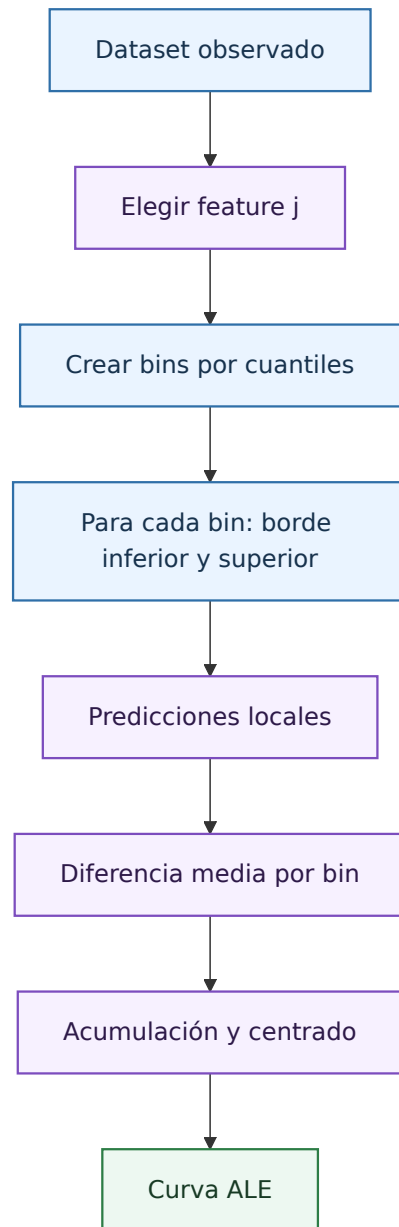


Figura 54: Construcción de curvas ALE por bins locales.

ALE debe interpretarse como efecto relativo acumulado, no como nivel absoluto.

- Si la curva ALE crece de forma sostenida, esa feature empuja la predicción al alza en las regiones observadas.
- Si la curva es casi plana, el efecto medio local es débil o está compensado entre regímenes distintos.
- Si aparecen cambios bruscos, conviene comprobar si corresponden a bins con poca densidad o a comportamiento realmente no lineal del modelo.

La complementariedad con ICE es clave.

- ICE muestra trayectorias individuales y, por tanto, heterogeneidad.
- ALE resume el efecto agregado dentro de zonas observadas y, por tanto, mejora la robustez de la lectura.

Si ambos apuntan en la misma dirección, la conclusión es más sólida. Si discrepan, el modelo probablemente está mezclando regímenes o usando interacciones que conviene revisar con superficies y diagnóstico por regiones.

ÁRBOLES SURROGATE Y REGRESIÓN SIMBÓLICA

Los modelos surrogate aproximan el comportamiento del modelo principal con una representación más interpretable. Formalmente, buscan una función h tal que:

$$\hat{\sigma}_{principal}(x) \approx h(x). \quad (22)$$

La variable objetivo de h no es la volatilidad real, sino la predicción del modelo principal. Por tanto, la fidelidad mide capacidad de imitación, no precisión frente a mercado. Esta distinción es esencial para evitar una interpretación incorrecta en auditoría.

Los árboles surrogate ofrecen reglas por particiones. Profundidades bajas facilitan lectura, pero pueden perder fidelidad; profundidades altas imitan mejor, pero se vuelven menos explicables. El dashboard conserva varios niveles de profundidad para elegir el equilibrio adecuado.

La regresión simbólica busca expresiones cerradas con operadores matemáticos. Su atractivo es producir fórmulas compactas que pueden discutirse financieramente. Su riesgo es doble: puede sobreajustar la predicción del modelo principal y puede proponer expresiones difíciles de justificar económicamente. Por ello se revisan conjuntamente error, complejidad y estabilidad de candidatos [Cranmer, 2023].

En el proyecto, los árboles se entrenan con `DecisionTreeRegressor` para profundidades 2, 4, 8 y 16. La configuración usa `surrogate_sample_size=12000` y `surrogate_min_samples_leaf=80`. La regresión simbólica usa PySR con una muestra de 2500 observaciones, búsqueda evolutiva, operadores binarios $+$, $-$, $\times$, $/$, $\wedge$, operadores unarios `square` y `cube`, límite de complejidad y `timeout`. En ambos casos, las métricas de fidelidad deben presentarse junto al error de mercado del modelo principal.

Este anexo distingue dos tipos de legibilidad.

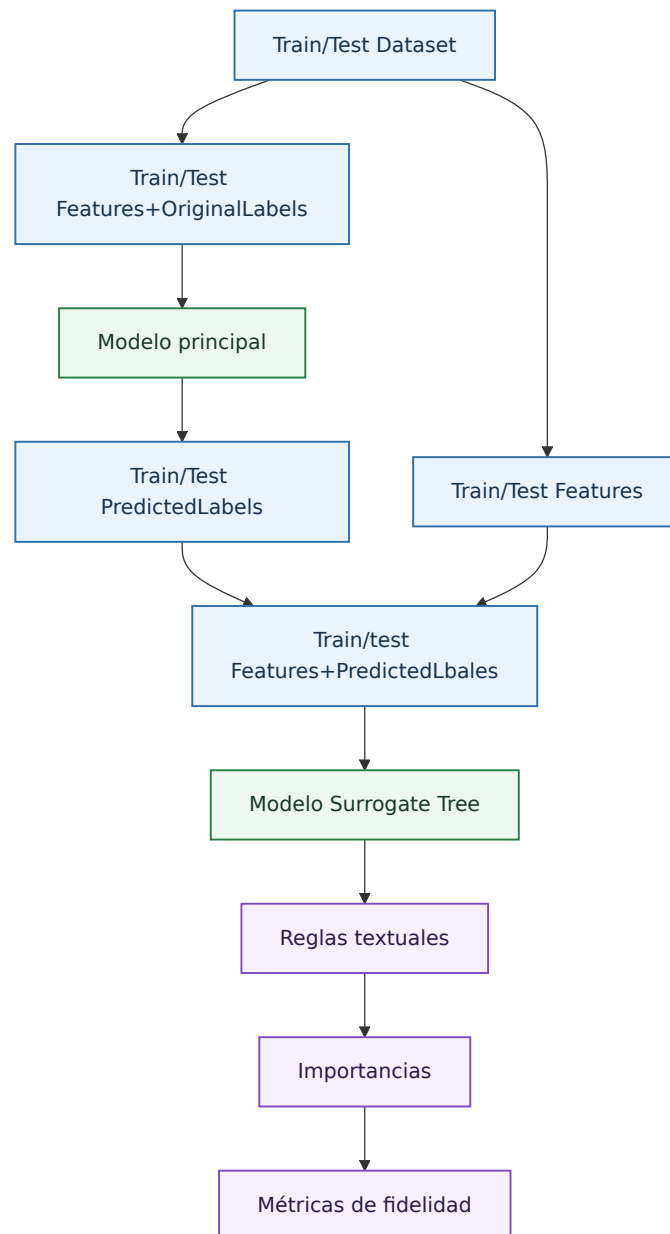


Figura 55: Entrenamiento y evaluación de arboles surrogate.

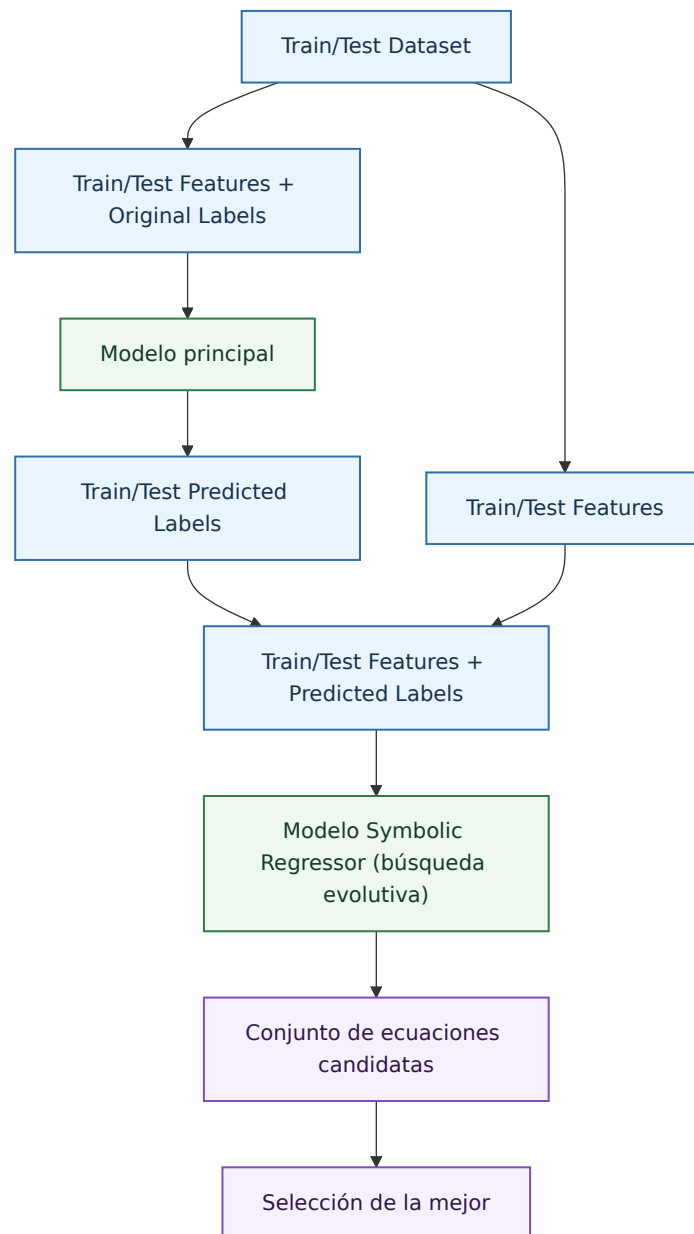


Figura 56: Entrenamiento del modelo simbolico surrogate.

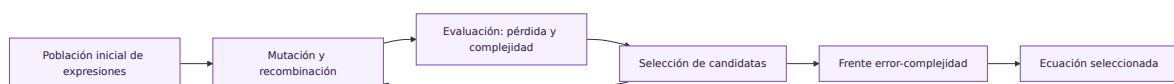


Figura 57: Búsqueda evolutiva de expresiones simbolicas.

- **Legibilidad por reglas:** propia de árboles surrogate, útil para umbrales y particiones del espacio explicativo.
- **Legibilidad algebraica:** propia de la regresión simbólica, útil cuando el comportamiento puede resumirse mediante una fórmula compacta.

La decisión de usar uno u otro depende del modelo final y de la fidelidad obtenida.

- Si un árbol poco profundo ya imita bien al modelo principal, la lectura por reglas puede ser suficiente.
- Si la fórmula simbólica logra buena fidelidad con complejidad moderada, aporta una síntesis especialmente valiosa.
- Si ambos fallan, la conclusión útil no es negativa, sino diagnóstica: el modelo principal necesita otras herramientas de explicación y no debe comprimirse artificialmente.

ENTRENAMIENTO PROGRESSIVE ATM

El entrenamiento progressive ATM se desarrolló como estudio paralelo para intentar mejorar el aprendizaje de la superficie. Su hipótesis de partida es financiera: la región ATM suele ser más líquida y central para la formación del nivel de volatilidad, mientras que las alas pueden contener más ruido, menor frecuencia o mayor sensibilidad a microestructura.

El procedimiento ordena las observaciones por distancia a ATM:

$$d_{ATM} = |\log(F/K)|. \quad (23)$$

Después divide el entrenamiento en segmentos. En la configuración actual se usan tres segmentos. Los modelos no iterativos reciben pesos mayores para regiones más ATM; los modelos iterativos entrenan de forma acumulativa, empezando por la zona central e incorporando progresivamente alas.

Si hay S segmentos ordenados de ATM a alas, los pesos conceptuales decrecen con el segmento:

$$w_s \propto S - s, \quad \tilde{w}_i = \frac{w_i}{\bar{w}}. \quad (24)$$

La diferencia por familia es relevante: regresión lineal y Random Forest usan pesos de muestra; XGBoost reparte estimadores entre fases acumulativas; y las redes entrenan sucesivamente sobre datasets acumulados. En los resultados de este trabajo, progressive ATM mejora Quantum Inspired pero no mejora al Random Forest seleccionado. Por tanto, se conserva como estrategia analítica útil, pero no como regla general de entrenamiento.

Desde el punto de vista operativo, el enfoque progressive ATM debe leerse como una hipótesis controlada.

- Hipótesis financiera: la zona ATM concentra parte relevante de la señal más líquida y estable.
- Hipótesis algorítmica: algunas arquitecturas pueden beneficiarse de aprender primero esa región central antes de absorber alas más ruidosas.
- Criterio de aceptación: sólo merece mantenerse si mejora test o si aporta una ventaja clara en suavidad y lectura de superficie sin degradar el resto.

En este trabajo, la evidencia es mixta.

- Resulta útil para la variante Quantum Inspired.
- Es prácticamente neutro en Random Forest.
- Deteriora el resultado final en otras familias como XGBoost o la red secuencial.

Eso obliga a una conclusión metodológica sobria: progressive ATM es una estrategia interesante para explorar, no una mejora universal ni una receta que deba imponerse a todas las familias.

BIBLIOGRAFÍA

- Daniel W. Apley and Jingyu Zhu. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society: Series B*, 82(4):1059–1086, 2020.
- Fischer Black. The pricing of commodity contracts. *Journal of Financial Economics*, 3(1-2):167–179, 1976.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- Miles Cranmer. Interpretable machine learning for science with PySR and symboli-cregression.jl. *arXiv preprint arXiv:2305.01582*, 2023.
- Emanuel Derman and Michael B. Miller. *The Volatility Smile*. Wiley, 2016.
- Alex Goldstein, Adam Kapelner, Justin Bleich, and Emil Pitkin. Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65, 2015.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 11 edition, 2022.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.